

Année universitaire 2025-2026

MEMOIRE DE STAGE

Mise en place d'outils pour la supervision du réseau

MESULOG SAS



Présenté par

Vianney MIQUEL

Jury

IUT : M. FONTENAS

IUT : M. DUTOT

Société : M. DESRUELLE

Déclaration de respect des droits d'auteurs

Par la présente, je déclare être le seul auteur de ce rapport et assure qu'aucune autre ressource que celles indiquées n'ont été utilisées pour la réalisation de ce travail. Tout emprunt (citation ou référence) littéral ou non à des documents publiés ou inédits est référencé comme tel et tout usage à un outil doté d'IA a été mentionné et sera de ma responsabilité.

Je suis informé qu'en cas de flagrant délit de fraude, les sanctions prévues dans le règlement des études en cas de fraude aux examens par application du décret 92-657 du 13 juillet 1992 peuvent s'appliquer. Elles seront décidées par la commission disciplinaire de l'UGA.

À Moirans

Le 15/06/2026

Signature



Remerciements

Ce stage a été une expérience très positive, où j'ai pu me tester et acquérir de nouvelles compétences, aussi bien techniques que professionnelles. Je tiens donc à remercier toutes les personnes qui ont rendu cette expérience intéressante ; et pour leurs conseils, soutien et accompagnement je souhaite remercier :

- **Luc Desruelle** Mon maître de stage qui m'a accueilli dans l'entreprise et m'a suivi tout au long de mon parcours. Il m'a permis de développer mes connaissances en aiguillant mes choix techniques
- **Benjamin Lerch** Développeur dans l'équipe Mesulog mais aussi responsable du service informatique, il a été d'une grande aide pour appréhender le système d'information de l'entreprise et m'a porté conseil en cas de problème.
- **Kenzo Sechi** Qui était aussi en alternance chez Mesulog, qui m'a aidé à m'intégrer dans l'équipe et a été d'une grande aide pour toute question.
- **Toute l'équipe Mesulog** Pour l'accueil à bras ouvert, qui m'a aidé à m'intégrer dans l'équipe. Pour leur aide et leurs conseils au long de mon stage.
- **Eric Fontenas** Mon tuteur pédagogique qui a répondu à toutes mes questions.
- **Pierre-françois Dutot** Qui a pris le temps de lire ce rapport afin de m'évaluer

Table des matières

1	Présentation du contexte professionnel	1
1.1	Présentation de l'entreprise	1
1.1.1	Présentation générale	1
1.1.2	Présentation du service d'accueil et de la hiérarchie Mesulog	2
1.1.3	Présentation de l'environnement technique	3
1.2	Objectifs et organisation des missions	4
2	Réalisation	5
2.1	Base de données Access	5
2.1.1	Nettoyage des données	7
2.1.2	Extraction des données pertinentes	8
2.2	Mise en place de Suricata	10
2.2.1	Configuration et intégration avec Wazuh	13
2.3	Installation et Intégration de Wazuh	15
2.3.1	Présentation de Wazuh et de son fonctionnement	16
2.3.2	Acquisition	17
2.3.3	Analyse	18
2.3.4	Wazuh API	21
2.4	Durcissement système et administration Windows Server	22
2.4.1	CIS Benchmark	22
2.4.2	Structure de l'Active Directory	22
2.4.3	Déploiement des GPOs	24
2.4.4	Exemples concrets de GPO appliquées	25
2.4.5	Suivi de l'évolution de la conformité	25
2.4.6	Limitation du module SCA de Wazuh	27
3	Impact environnemental et sociétal	29
3.1	Déplacements	29
3.2	Répercussions écologiques et sociales des actions menées par l'entreprise	29
4	Conclusion	30
5	Annexe	31
	Références	53

Table des figures

1	Présentation des sociétés filles de Nerys Group et de leurs directions	1
2	Présentation de la hiérarchie de l'équipe Mesulog	2
3	Plan réseau de Mesulog en 2026	3
4	Chronologie des missions réalisées pendant mon stage	5
5	Schéma de la table initiale	6
6	Requête de normalisation des fins de numéro de série	7
7	Tableau récapitulatif général - (pour le banc VM-TD14-BHT)	8
8	Requête d'une table temporaire pour le calcul du TtFP (Time to First Pass)	9
9	Distribution des statuts de fin des essais par étape de l'essai, tous bancs confondus	10
10	Exemple de règle Suricata - ici une règle contre le mDNS poisoning	10
11	fichier Docker Compose	11
12	Interface web d'Evebox	12
13	Dashboard d'événement Docker dans Wazuh	13
14	Programme de mise à jour automatique des règles d'ip malicieuses	14
15	Timer pour le service linux suricata-ip-update	15
16	Service Linux suricata-ip-update	15
17	Schéma de l'architecture de Wazuh	16
18	Chaîne de traitement d'un log dans Wazuh	17
19	Extrait du fichier de configuration "ossec.conf" du Manager Wazuh	17
20	Paramétrage d'envoi du journal Sophos	18
21	Exemple de log brut Unify - connexion établie	18
22	Décodeur de log de connexion au réseau Wifi	19
23	Règles de connexion au réseau Wifi	19
24	Interface web de Wazuh regroupant les règles activées	20
25	Règle wazuh pour remonter l'alerte de l'intégration Python	21
26	Organisation initiale de l'Active Directory	22
27	Organisation de l'Active Directory après restructuration	23
28	Extrait de la 2ème itération de déploiement	25
29	Commande gpupdate pour forcer la récupération des GPO sur le poste HP026	25
30	Conformité du poste HP026 avant la première itération de déploiement des GPO	26
31	Évolution de la conformité CIS Level 1 du poste HP026 sur 4 itérations	26
32	Extrait d'un rapport d'application des GPO sur le poste HP026	27
33	Test manuel d'une commande utilisée par l'agent, sur le poste HP026	27
34	Commande PowerShell modifiée pour supporter les accents	28
35	Extrait de la GPO permettant de déployer le fichier de test modifié sur les postes	28
36	Requêtes de nettoyage des données -1ère partie	31
37	Requêtes de nettoyage des données -2ème partie	32
38	Requêtes de nettoyage des données -3ème partie	33
39	Requêtes de nettoyage des données -4ème partie	34
40	Requêtes d'extraction de données -1ere partie	35
41	Requêtes d'extraction de données -2eme partie	36
42	Requêtes d'extraction de données -3eme partie	37
43	Requêtes d'extraction de données -4eme partie	38
44	Requêtes d'extraction de données -5eme partie	39
45	Requêtes d'extraction de données -6eme partie	40
46	Requêtes d'extraction de données -6eme partie	41
47	Requêtes d'extraction de données -7eme partie	42
48	Requêtes d'extraction de données -8eme partie	43

49	Requêtes d'extraction de données -9eme partie	44
50	Script sophos-exfiltration.py -1ere partie	45
51	Script sophos-exfiltration.py -2ème partie	46
52	Script sophos-exfiltration.py -3eme partie	47
53	Script sophos-exfiltration.py -4eme partie	48
54	Distribution des statuts de fin des essais par étape de l'essai, modules V2	49

Résumé court

Ce rapport présente la réalisation de plusieurs missions au sein de l'entreprise Mesulog durant un stage. Ces missions sont en lien avec le domaine de l'administration système et réseau. Les missions sont détaillées sous forme d'explication des étapes par ordre chronologique.

Keywords : *administration, système, réseau, stage.*

Abstract

This report presents different missions of a 3 and a half internship. This end-of-degree internship was located at the company Mesulog, in Moirans. The main subject of this internship is system and network administration, with an emphasis on the cybersecurity stakes of these missions. The main assignments were data processing and analysing to find key performance indicators, the installation and configuration of an IDS solution, feeding a SIEM installation, which centralize logs and the monitoring of network's events; and global system hardening using GPO and Windows Server administration. This report outlines the various stages of the process, the challenges encountered, and the solutions that were found by chronological order. This internship provided real-world first-hand experience, and a broad range of subjects to learn from.

Keywords : internship, security, network, monitoring

Introduction

Dans le cadre de mon BUT Informatique à l'IUT2, j'ai pu effectuer mon stage de fin de 3ème année dans l'entreprise Mesulog. Mesulog est une entreprise spécialisée dans les systèmes de test et mesures pour banc de tests. Cet environnement d'entreprise tourné vers l'industrie apporte des défis particuliers auxquels je dois me confronter.

Ce stage s'inscrit donc dans un processus de sécurisation du système d'information de l'entreprise, essentiel afin de garantir notamment la confidentialité de données industrielles confidentielles. La mise en place d'outils de supervision et de management d'événements de réseau et de sécurité a donc été mon sujet central au cours de ces 14 semaines.

Pour mieux comprendre les missions qui m'ont été confiées, il est important de présenter le contexte professionnel dans lequel j'ai évolué au cours de mon stage. La première partie de ce rapport est donc dédiée à la présentation de l'entreprise et du service d'accueil.

1 Présentation du contexte professionnel

1.1 Présentation de l'entreprise

1.1.1 Présentation générale

Mesulog est une entreprise basée à Moirans, en Isère. Elle est spécialisée dans le développement de logiciels : séquenceurs de test et outils de mesure ; pour des bancs de test de produits, dans un domaine industriel. Avec de nombreuses certifications, Mesulog se place comme une entreprise spécialiste dans son domaine. C'est donc une PME avec des clients importants qui évoluent dans différents secteurs, allant de l'aéronautique à la recherche, en passant par l'industrie et l'automobile.

Mesulog fait aussi partie d'un groupe d'entreprise, **Nerys Group**. Nerys Group est la société mère qui regroupe Mesulog en Isère, Nerys dans les Bouches-du-Rhône et Territoire de Changements, ensuite devenue Nerys Group Service, en Savoie. Tandis que Mesulog est centrée autour du développement logiciel, Nerys est responsable de la conception de bancs de test, baies et coffrets électriques pour les environnements industriels. Nerys Group Service est quant à elle axée sur la proposition de formation dans les outils NI Labview et NI TestStand, logiciels utilisés pour la supervision des bancs et séquencer les tests.

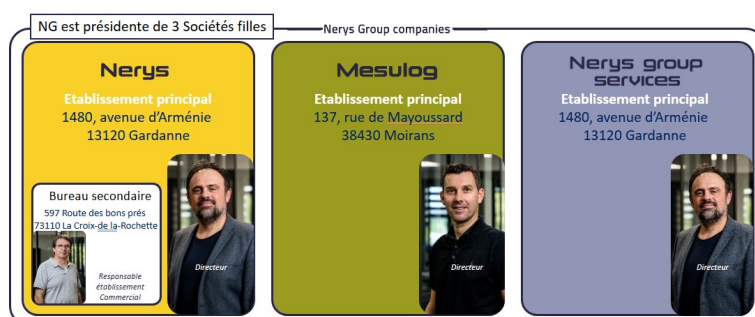


FIGURE 1 – Présentation des sociétés filles de Nerys Group et de leurs directions

1.1.2 Présentation du service d'accueil et de la hiérarchie Mesulog

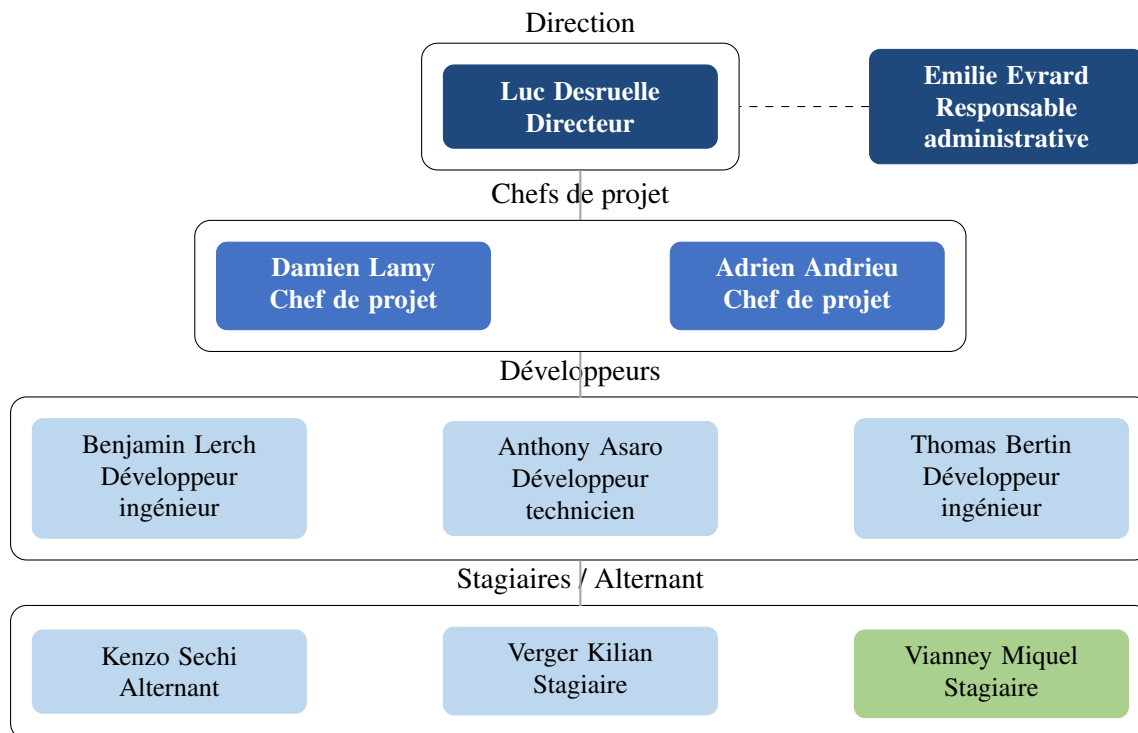


FIGURE 2 – Présentation de la hiérarchie de l'équipe Mesulog

Nous pouvons observer ci-dessus l'organigramme de l'équipe de Mesulog. L'équipe est composée de **Luc Desruelle**, qui est président de Nerys Group et assure le poste de directeur de Mesulog. Il est aussi le DSI de Mesulog.

Nous avons ensuite **Damien Lamy** et **Adrien Andrieu**, chefs de projet. Ils ont aussi des compétences de développeurs et présentent un profil technico-commercial. Le reste de l'équipe est composée de développeurs, à noter que Benjamin Lerch est également le RSSI du service informatique, il est donc mon supérieur direct.

Nous avons ensuite **Kenzo Sechi**, alternant en informatique, qui travaille avec moi. Le stagiaire **Kilian Verger** est un stagiaire en BUT mesures physiques.

Emilie Evrard est quant à elle la responsable administrative de Nerys Group. Les activités de comptabilité et de ressources humaines étant mutualisées entre les entreprises du groupe. Certains acteurs extérieurs à l'équipe de Mesulog travaillent de concert avec l'équipe, notamment des salariés de Nerys et Nerys Groups Service. Cependant ils ne sont pas représentés sur ce schéma organisationnel car je n'ai aucun contact avec eux dans mes missions.

Je suis donc entouré d'une équipe expérimentée qui peut m'apporter de l'aide et des conseils en cas de problème. J'ai aussi un contact direct avec mes supérieurs que ce soit Benjamin Lerch ou Luc Desruelle, mon maître de stage. Cela me permet de bénéficier d'un suivi régulier dans mes missions.

1.1.3 Présentation de l'environnement technique

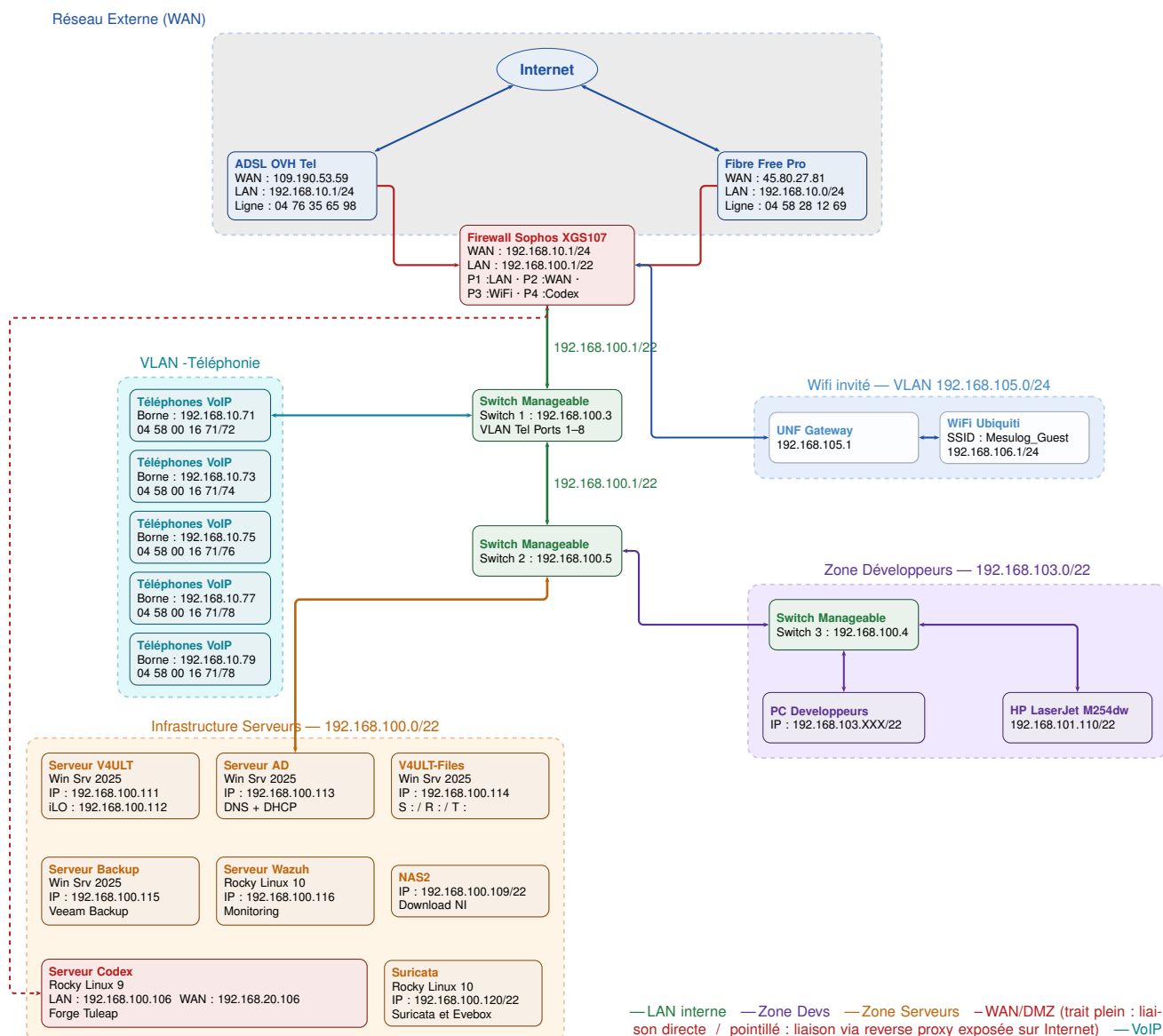


FIGURE 3 – Plan réseau de Mesulog en 2026

Voici l'architecture réseau de Mesulog. Le réseau comporte un VLAN pour la téléphonie, un LAN physique pour le wifi et des zones d'adressages, une pour les serveurs et une pour les ordinateurs des employés. Il n'y a donc pas de segmentation réseau réelle entre la zone serveur et la zone employés. Nos serveurs sont hébergés sur un hyperviseur Hyper-V, Serveur V4ULT, à l'exception du serveur Suricata et du NAS.

Les serveurs de l'infrastructure sont regroupés sous le nom de domaine interne **V4ULT**, on y retrouve notamment :

- **V4ULT-AD** : l'Active Directory (Serveur **LDAP**) faisant aussi office de Serveur DHCP et DNS.
- **V4ULT-FILES** : Le serveur de fichiers de l'entreprise découpés en 3 entités (Echanges, Mesulog, Downloads)

- **Server Backup** : Le serveur de sauvegarde de l'entreprise utilisant Veeam Backup & Replication.
- **Server Codex** : Instance de Tuleap, similaire à GitLab, gestionnaire de version de code source basé SVN. Le serveur est exposé sur internet derrière un reverse proxy et un firewall.

Quant à l'environnement logiciel, les utilisateurs, principalement des développeurs NI LabVIEW & NI TestStand, utilisent Windows 11 avec la suite de logiciels suivante :

- **Oracle VirtualBox** : virtualisation pour exécuter des machines virtuelles locales.
- **Suite Microsoft 365** : bureautique et collaboration (Outlook, Word, Excel, Teams...).
- **Firefox** : navigateur web principal pour la navigation et les outils web internes.
- **TeamViewer** : assistance à distance et support technique.

Les logiciels utilisés dans les machines virtuelles peuvent être les suivants :

- **NI LabVIEW** : environnement de développement pour les applications de test et mesure.
- **NI TestStand** : Séquenceur de tests automatisés.
- **Logiciels propriétaires clients** : applications spécifiques fournies par des clients (e.g. ?)

1.2 Objectifs et organisation des missions

Durant mon stage, j'ai pu effectuer plusieurs missions, certaines étaient prévues à l'avance, d'autres sont apparues au fur et à mesure de l'avancement du stage. L'objectif initial de ce stage était la mise en place d'outils de supervision et de management d'événements de sécurité, tels que Suricata et Wazuh.

Cependant ce n'est pas la première mission que j'ai effectuée, en effet pour appréhender ce nouvel environnement j'ai commencé par une mission d'analyse et de mise en forme de données sur une base de données Access.

J'ai ensuite mis en place Suricata, un IDS open source, afin de surveiller les événements réseau. Suite à quoi j'ai installé et configuré le SIEM open-source Wazuh.

J'ai aussi pu retravailler l'organisation de l'Active Directory de l'entreprise, avec des mises en place de GPO, ou encore participé à la création d'une campagne interne de test de phishing. Certaines plus petites tâches se sont intercalées entre ces sujets principaux, elles ne seront pas détaillées dans ce rapport.

Ces missions étant toutes étroitement liées entre elles, excepté l'analyse de données pour un client, elles se sont déroulées en parallèle sur la durée de mon stage.

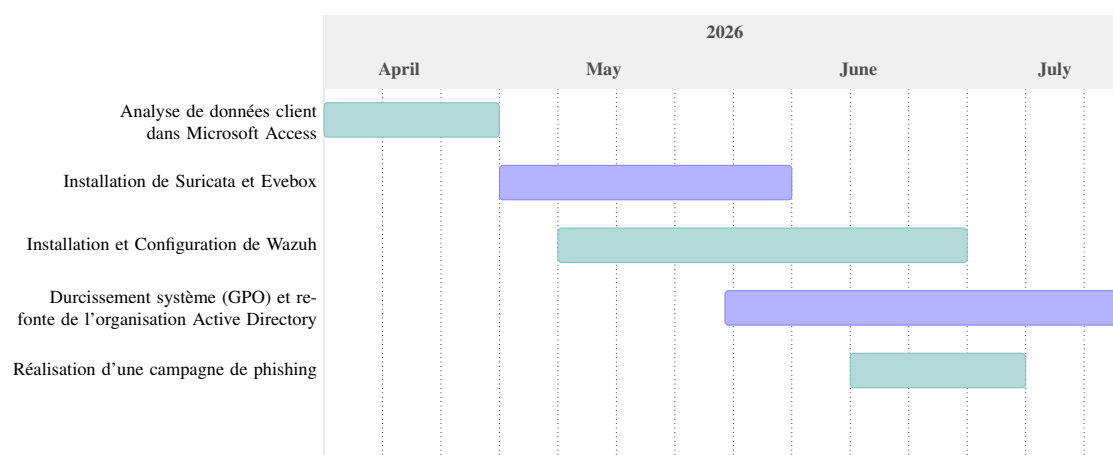


FIGURE 4 – Chronologie des missions réalisées pendant mon stage

2 Réalisation

2.1 Base de données Access

Au début de mon stage, j'ai travaillé sur une analyse de base de données Microsoft Access pour un client de Mesulog. Cette base de données, générée automatiquement depuis les rapports de test de banc de test Mesulog présent chez des clients, se présente au départ comme 1 seule table à 23 colonnes.

Field Name	Data Type
StartTime	Date/Time
FolderStatus	Short Text
ReportFileName	Short Text
ReportPath	Short Text
UHF100name	Short Text
UUTresult	Short Text
StepCount	Number
StationID	Short Text
SerialNumber	Short Text
LoginName	Short Text
TotalTime	Short Text
Model	Short Text
Hardware	Short Text
PowerSupply	Short Text
DigitalInputVoltage	Short Text
PDTimerSynchronizationReferen	Short Text
UHFspectrumBandwith	Short Text
UHFspectrumResolution	Short Text
TimeAnalysisSamplingFrequency	Short Text
TimeAnalysisBandwith	Short Text
DallasSerialNumber	Short Text
MACaddress	Short Text
SoftwareVersion	Short Text
FirmwareVersion	Short Text

FIGURE 5 – Schéma de la table initiale

Il était donc courant d'avoir des enregistrements partiels, erronés ou avec un mauvais format. L'objectif final était de produire différents graphiques et tableaux pour analyser l'évolution de la production de modules, appelés "V1" ou "V2".

Les modules sont testés sur un banc, ils sont soit de type 1 ou 2. Un module passe un ou plusieurs tests ou essais avant d'être validé. Pour qu'un module soit validé il suffit d'un essai passé. Un essai à 4 issues : "Passed" (valide), "Failed" (non-valide), "Terminated" (le test s'est terminé prématurément) et "Error" (le banc de test a fait crasher le test).

Nous devons donc produire un tableau récapitulatif général par année et par banc avec entre autre les nombres de modules validés, le nombre d'essais moyen et médian avant pour un module avant d'être validé, le temps moyen en heure pour valider un module, le ratio des états des essais.

Le client a aussi 2 demandes plus particulières :

- Nous devons analyser plus précisément les états des essais les lundi matins car un nombre anormal d'erreurs sur le banc est suspecté.
- Nous devons aussi analyser la distribution des états des essais selon le pas de test. En effet une séquence d'essais complète comporte 39 étapes.

2.1.1 Nettoyage des données

Pour pouvoir correctement exploiter les données, il est important d'avoir des données complètes et sous le même format.

Les colonnes comportant des erreurs ou des oublis sont principalement :

- Le numéro de série du module, qui comporte parfois des espaces, lettres minuscule ou un mauvais nombre de caractères.
- Le type du module (V1 ou V2), qui est noté UHF100 pour V1, UHF100 V2 ou UHF1002Ghz. Cette valeur est souvent manquante, il faut donc l'inférer.
- Les essais qui font partie de la chaîne production et les essais de R&D ou de SAV qui ne sont pas signalés de manière consistante. Il faut écarter ces enregistrements de l'analyse.

Pour ce faire nous avons créé une suite de requêtes SQL pour automatiquement effectuer ce nettoyage. Étant donné que l'objectif final est d'automatiser l'exécution de toutes ces requêtes dans Microsoft Access avec un script VBS (Visual Basic Scripting), la syntaxe SQL utilisée est celle de Microsoft Access, qui nous impose des choix parfois sous-optimaux.

Pour ne pas surcharger ce rapport nous verrons ici seulement un exemple de formatage, le reste est disponible en [Annexe](#) :

```
1  --SETUP-09-SerialNumberNormalise-3
2
3  UPDATE RAW_DATA
4  SET
5      SerialNumberNormalise = IIf(
6          InStr (SerialNumberNormalise, ".") > 0,
7          Left (
8              SerialNumberNormalise,
9              InStr (SerialNumberNormalise, ".") -1
10         ) & "." & Right (
11             "000" & Mid (
12                 SerialNumberNormalise,
13                 InStr (SerialNumberNormalise, ".") + 1
14             ),
15             3
16         ),
17         SerialNumberNormalise
18     )
19 WHERE
20     InStr (SerialNumberNormalise, ".") > 0;
```

FIGURE 6 – Requête de normalisation des fins de numéro de série

Nous pouvons voir ci-dessus une requête destinée à formater tous les numéros de série sous la forme "*.XXX", "XXX" étant 3 chiffres.

Exemple : "17A240923.2" est transformé en "17A240923.002"

Afin de ne perdre aucune information, le numéro de série normalisé est inséré dans une nouvelle colonne. Il en va de même pour le modèle et pour l'objectif du test (Production ou SAV, R&D).

Une fois que toutes les données sont complétées et formatées, nous passons à l'exploitation de celles-ci.

2.1.2 Extraction des données pertinentes

L'objectif de cette partie est de remplir un tableau type comme présenté ci-dessous :

Banc 2 VM-TD14-BHT									
Année	2020	2021	2022	2023	2024	2025	2026	total	
Nb modules testés au Total	0	0	0	0	76	311	25	412	
Nb modules testés Production	0	0	0	0	75	310	20	405	405
Nb modules Prod validés	0	0	0	0	71	300	16	387	96 %
Nb modules Prod Non validés	0	0	0	0	4	10	4	18	4 %
Ratio module V2 sur total (%)					100,00 %	100,00 %	100,00 %		
Effort Pass (Moyenne)					3,65	2,73	3,44	3,3	
Effort Pass (Mediane)					2,00	2,00	3,00	2,3	
Time to First Pass (TtFP) en Heure					3,44	3,08	3,17		
Nb Essais Prod total					311	866	79	1256	31 %
Ratio Nb Essai Passed					23,47 %	34,76 %	18,99 %	25,74 %	
Ratio Nb Essai Failed					3,86 %	6,58 %	3,80 %	4,75 %	
Ratio Nb Essai Terminated					63,99 %	51,73 %	73,42 %	63,05 %	
Ratio Nb Essai Error					8,68 %	6,93 %	3,80 %	6,47 %	
Nb Essai Erreur Lundi 6h-10h					2	16	1	6,3	
Ratio Essai Erreur Lundi 6h-10h					10,00 %	17,39 %	12,50 %	13,30 %	
Nb Essai Erreur MMJV/4 6h-10h					2,75	4,00	0,00	2,3	
Ratio Essai Erreur MMJV/4 6h-10h					10,28 %	5,25 %	0,00 %	6,38 %	

FIGURE 7 – Tableau récapitulatif général - (pour le banc VM-TD14-BHT)

Ce tableau récapitulatif permet de voir facilement l'évolution de l'utilisation du banc, des résultats d'essais et du nombre de modules sortis en production.

Pour plus de précision, le "Effort to Pass" est le nombre d'essais avant une réussite, tandis que le "Time to First Pass" est le temps cumulé de tous les essais jusqu'à un essais validant.

Nous pouvons aussi voir des lignes mettant en évidence les différences d'essais en erreur entre les lundis matins et les autres jours de la semaine.

Dans ce jeu de données, nous avons 2 bancs et 2 types de modules. Il était demandé d'analyser l'évolution de la production par banc tous types de modules confondus et par banc seulement pour les modules types V2. Cela nous fait donc 6 tableaux à remplir de données.

Pour extraire ces données, nous partons de la table nettoyée, et nous avons fait une suite de 18 requêtes SQL pour extraire les données ou créer des tables temporaires pour d'autres requêtes. Si dans ces requêtes nous définissons le banc et le type de module en dur, il est prévu d'utiliser un script VBS (Visual Basic Scripting) pour remplacer automatiquement ces variables dans les requêtes.

Comme précédemment, nous ne montrons pas l'intégralité des requêtes dans le corps du rapport, mais elles sont disponibles en [ici](#).

```
1      Round (
2      (
3      (
4          IIf (
5              Min(IIf(Passed = True, StartTime, NULL)) IS NULL,
6              NULL,
7              DateDiff(
8                  "s",
9                  Min(StartTime),
10                 Min(IIf(Passed = True, StartTime, NULL))
11             )
12         ) + Min(IIf(Passed = True, TotalTime, NULL))
13     ) / 3600
14 ),
15 3
16 ) AS TimeToFirstPassHours
17 INTO
18     B1_temp_timediff
19 FROM
20     B1_table_essais_statut
21 GROUP BY
22     SerialNumberNormalise;
23
24 --BC-MOD-16-TempSortedTimediff
25 SELECT DISTINCT
26     SerialNumberNormalise,
27     TimeToFirstPassHours,
28     Year(FirstPass) AS Annee
29 INTO
30     B1_temp_sorted_TimeDiff
31 FROM
32     B1_temp_timediff
33 WHERE
34     TimeToFirstPassHours IS NOT NULL
35 ORDER BY
36     Year(FirstPass),
37     TimeToFirstPassHours;
```

FIGURE 8 – Requête d'une table temporaire pour le calcul du TtFP (Time to First Pass)

Cette requête crée une table temporaire qui regroupe l'empreinte temporelle du premier essai d'un module, du premier essai validant, de la différence entre ces deux essais, du temps d'essai du second essai et finalement du "Time to First Pass" arrondi à l'heure.

Maintenant que nous avons extrait toutes les données nécessaires, nous pouvons les mettre en forme, voici donc quelques exemples de graphiques produits (L'ensemble est trouvable en [Annexe](#)) :

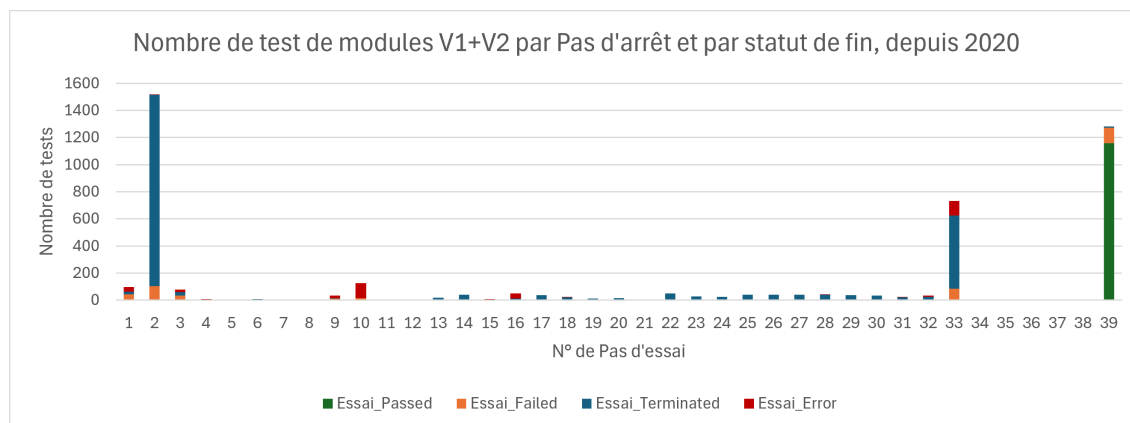


FIGURE 9 – Distribution des statuts de fin des essais par étape de l'essai, tous bancs confondus

2.2 Mise en place de Suricata

Suricata est un système de détection/prévention d'intrusion. Son fonctionnement est assez simple : il capture chaque paquet qui transite sur le réseau et analyse ses propriétés. Pour engendrer une alerte ou une action, Suricata utilise des règles, qui sont un ensemble de paramètres. Si une trame correspond à tous les critères (elle a la même signature) alors une action est prise par Suricata.

```
1 alert udp any any -> any 546 (msg:"Suspicious DHCPv6 Advertise Packet
   Detected, Possible attack using mitm6"; content:"|02|"; depth:1;
   offset:0; reference:url,https://blog.exploit.org/caster-
   neurotransmitter/; sid:1118750; rev:1;)
```

FIGURE 10 – Exemple de règle Suricata - ici une règle contre le mDNS poisoning

Suricata peut donc décider de ne rien faire, de générer une alerte ou de rejeter la trame en mode IPS (Intrusion Prevention System). En mode IDS (Intrusion Detection System) nous lançons simplement des alertes mais sans bloquer le trafic. C'est comme ça que nous l'installerons, afin d'éviter qu'un faux positif coupe une connexion légitime.

Pour installer Suricata, nous avons pris une machine physique. Pour éviter de risquer de ralentir le trafic, nous avons mis en place un port mirroring à partir d'un switch juste en aval du pare-feu Sophos, le Suricata reçoit donc une copie exacte de tout le trafic qui passe de WAN à LAN et de LAN à WAN.

La distribution Linux utilisée en production à Mesulog est Rocky Linux 10, une distribution RHEL (Red Hat). L'installation de Suricata sur Rocky Linux se fait avec le dépôt COPR du projet OISF (développeurs et mainteneurs de Suricata).

Cependant ce dépôt qui installe via un fichier rpm n'arrive pas à résoudre ses dépendances sur Rocky 10. Nous avons donc choisi d'installer Suricata via l'image Docker fournie par Jason Ish, membre du projet OISF.

Ce choix technique nous permet aussi d'installer Evebox dans un second conteneur Docker. Evebox est l'outil de gestion d'alerte et d'événement pour Suricata développé et maintenu lui aussi par le projet OISF. Ces 2 conteneurs sont réunis dans un même fichier docker compose pour une meilleure maintenabilité.

```
1  services:
2    suricata:
3      image: jasonish/suricata:latest
4      container_name: suricata
5      restart: always
6      network_mode: host
7      environment:
8        - TZ=Europe/Paris
9      cap_add:
10       - NET_ADMIN
11       - NET_RAW
12       - SYS_NICE
13      volumes:
14       - /var/log/suricata:/var/log/suricata
15       - /etc/suricata:/etc/suricata
16       - /var/lib/suricata/rules:/var/lib/suricata/rules
17      command:
18       - -i
19       - enp1s0
20       - -i
21       - enp2s0
22       - -c
23       - /etc/suricata/suricata.yaml
24    evebox:
25      image: jasonish/evebox:latest
26      container_name: evebox
27      restart: always
28      environment:
29        - TZ=Europe/Paris
30        - EVEBOX_RETENTION_PERIOD=168h
31      depends_on:
32        - suricata
33      ports:
34        - "5636:5636"
35      volumes:
36       - /var/log/suricata:/var/log/suricata
37      command:
38       - evebox
39       - server
40       - --host=0.0.0.0
41       - --port=5636
42       - --sqlite
43       - --input
44       - /var/log/suricata/eve.json
```

FIGURE 11 – fichier Docker Compose

Premièrement nous pouvons voir que les deux conteneurs sont paramétrés pour redémarrer automatiquement dès qu'ils sont éteints, pour limiter le temps hors-service. La variable d'environnement "TZ" permet de s'assurer que l'heure soit bien synchronisée dans les logs. Quelques points importants :

- Des capacités ont été ajoutées avec "cap_add" au conteneur Suricata pour accéder à l'interface réseau de la machine hôte, créer des sockets ou changer la priorité des processus si besoin.
- La variable d'environnement "EVEBOX_RETENTION_PERIOD" permet de définir une limite de temps au-delà de laquelle les logs sont effacés, très important pour ne pas saturer le stockage. Ici la limite est de 7 jours.

Ci-dessous l'interface web de Evebox, qui permet de suivre pas seulement les alertes mais tous les événements Suricata :

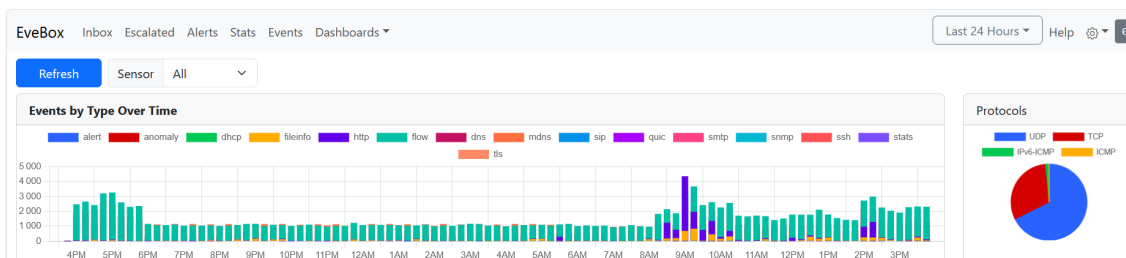


FIGURE 12 – Interface web d'Evebox

2.2.1 Configuration et intégration avec Wazuh

Pour éviter de surcharger le SIEM, nous avons créé un deuxième fichier de log sous le format EVE (Extensible Event Format), intitulé `eve-wazuh.json`, dans lequel seulement les alertes Suricata sont enregistrées, contrairement au fichier de log `eve.json` pour Evebox où toutes les alertes ainsi que tous les événements sont consignés.

Ainsi nous avons un agent wazuh sur la machine hôte Rocky linux de suricata et evebox. Cet agent est configuré pour utiliser le fichier `eve-wazuh.json` comme source.

Cet agent wazuh a aussi une deuxième fonction ; en effet il vérifie régulièrement l'état des conteneurs dockers, et remonte des alertes dans wazuh dès qu'un changement d'état ou une commande docker est lancée sur la machine.

Il suffit par la suite de créer des règles dans Wazuh pour attraper les alertes suricata pertinentes.

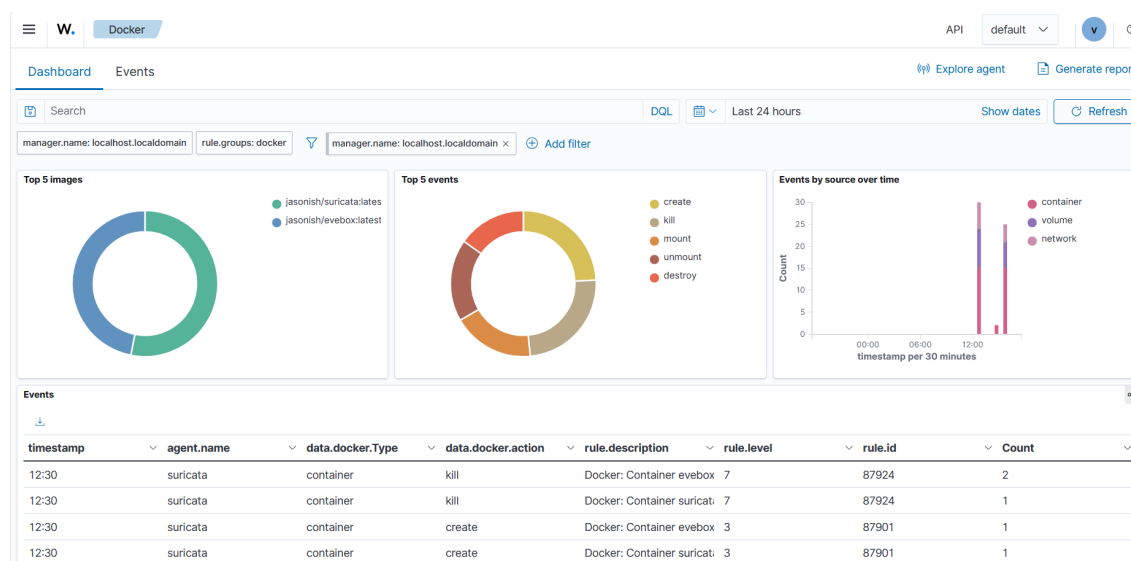


FIGURE 13 – Dashboard d'événement Docker dans Wazuh

En ce qui concerne les règles de Suricata en mode IDS, nous avons premièrement ajouté les règles de l'Open Emerging Threats publiées par Proofpoint. Avec des règles additionnelles, ajoutées à la suite de l'analyse de l'activité réseau avec Evebox.

Par exemple, en constatant un trafic significatif de paquets mDNS sur le réseau, des règles contre le mDNS poisoning ont été ajoutées dans un fichier de règle supplémentaire.

Afin de se prémunir contre les IP malicieuses, nous avons ajouté des règles automatiquement suivant une liste d'IP malicieuses.

Pour ce faire, nous avons écrit un programme bash qui récupère des listes d'IP malicieuses depuis des listes de emerging threats, abuse.ch ou encore alien vault (Voir annexe). Nous concaténons ensuite ces listes dans un seul fichier, puis ajoutons pour chaque IP 2 règles dans un fichier de règle Suricata "ip-reputation.rules" une règle pour les connexions sortantes et la deuxième pour les connexions entrantes depuis/vers cette IP. Ensuite le programme exécute "docker kill – signal=USR2 suricata" qui envoie un signal non pas d'arrêt mais de rechargement des règles à chaud sans redémarrer le service.

Ce rechargement est d'ailleurs détecté par l'agent Wazuh présent sur la machine hôte, qui remonte un événement de rechargement dans Wazuh.

```

1  #!/bin/bash
2  # /usr/local/bin/suricata-ip-update.sh
3
4  # Configuration et initialisation
5  RULES_DIR="/var/lib/suricata/rules"
6  CUSTOM_LIST="/etc/suricata/custom-blocklist.txt"
7  LOG_FILE="/var/log/suricata/threat-feeds-update.log"
8  SID_COUNTER=67000000
9
10 touch $LOG_FILE
11
12 echo "$(date): Starting threat feed update" >> $LOG_FILE
13
14 # Supprimer l'ancienne liste d'ip
15 > $CUSTOM_LIST
16
17 # 1. Emerging Threats Compromised IPs
18 echo "Fetching Emerging Threats compromised IPs..." >> $LOG_FILE
19 curl -s https://rules.emergingthreats.net/blockrules/compromised-ips.txt >> $CUSTOM_LIST
20
21 # 2. Feodo Tracker (abuse.ch)
22 echo "Fetching Feodo Tracker C2 IPs..." >> $LOG_FILE
23 curl -s feodotracker.abuse.ch/downloads/ipblocklist.csv | \
24 grep -v "^#" | cut -d',' -f1 >> $CUSTOM_LIST
25
26 # 3. Open Threat Exchange (OTX) AlienVault
27 echo "Fetching AlienVault OTX pulses..." >> $LOG_FILE
28 curl -s https://reputation.alienvault.com/reputation.generic | \
29 grep -v "^#" | awk -F ' #' '{print $1}' >> $CUSTOM_LIST
30
31 # 4. Greensnow
32 echo "Fetching Greensnow list..." >> $LOG_FILE
33 curl -s https://blocklist.greensnow.co/greensnow.txt | \
34 grep -E "^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$" >> $CUSTOM_LIST
35
36 # Supprimer les doublons
37 sort -u $CUSTOM_LIST -o $CUSTOM_LIST
38
39 # Total nombre ip
40 IP_COUNT=$(wc -l < $CUSTOM_LIST)
41 echo "$(date): Added $IP_COUNT unique IPs to blocklist" >> $LOG_FILE
42 echo "Converting IPs to Suricata rules..." >> $LOG_FILE
43
44 > "$RULES_DIR/ip-reputation.rules"
45
46 while IFS= read -r ip
47 do
48     # ignorer les lignes vides ou comentees
49     [[ -z "$ip" || "$ip" == \#* ]] && continue
50
51     SID_COUNTER=$((SID_COUNTER + 2))
52     SID_OUTBOUND=$((SID_COUNTER + 1))
53
54     echo "alert ip $ip any -> any any (msg:\\"REPUTATION Known malicious IP (inbound)- $ip\\"; reference:url,
55         emergingthreats.net; classtype:bad-unknown; sid:$SID_COUNTER; rev:1;)" >> \
56         "$RULES_DIR/ip-reputation.rules"
57
58     echo "alert any any -> ip $ip any (msg:\\"REPUTATION Known malicious IP (outbound)- $ip\\"; reference:url,
59         emergingthreats.net; classtype:bad-unknown; sid:$SID_OUTBOUND; rev:1;)" >> \
60         "$RULES_DIR/ip-reputation.rules"
61 done < $CUSTOM_LIST
62
63 echo "$(date): Generated $(wc -l < $RULES_DIR/ip-reputation.rules) rules" >> $LOG_FILE
64
65 docker kill --signal=USR2 suricata && \
66     echo "$(date): Reload signal sent successfully" >> $LOG_FILE || \
67     echo "$(date): ERROR - Suricata container is not running!" >> $LOG_FILE
68
69 echo "$(date): Update complete" >> $LOG_FILE
70 exit 0

```

FIGURE 14 – Programme de mise à jour automatique des règles d'ip malicieuses

Pour exécuter ce programme de manière automatique, nous avons créé un service Linux "suricata-ip-update.service" qui est lancé tous les jours à minuit par "suricata-ip-update.timer". Grâce à l'option "RandomizedDelaySec=1800", ce timer a une fenêtre de 30 min (entre 00 :00 et 00 :30) pour lancer le service, ce qui permet d'éviter de surcharger la machine si des processus lourds sont déjà en train de s'exécuter (sauvegardes automatiques entre autre).

```

1 [Unit]
2 Description=Run Suricata feed update daily
3 Requires=suricata-ip-update.service
4
5 [Timer]
6 OnCalendar=daily
7 Persistent=true
8 RandomizedDelaySec=1800
9
10 [Install]
11 WantedBy=timers.target

```

FIGURE 15 – Timer pour le service linux suricata-ip-update

```

1 [Unit]
2 Description=Update Suricata IP reputation feeds
3 After=network.target docker.service
4 Requires=docker.service
5
6 [Service]
7 Type=oneshot
8 ExecStart=/usr/local/bin/suricata-ip-update.sh
9 StandardOutput=journal
10 StandardError=journal
11 User=root
12
13 [Install]
14 WantedBy=multi-user.target

```

FIGURE 16 – Service Linux suricata-ip-update

2.3 Installation et Intégration de Wazuh

En parallèle de l'installation de Suricata, nous avons mis en place un SIEM (Outil de gestion d'événements et d'informations de sécurité). Nous avons opté pour le SIEM open source Wazuh. En effet, il est important pour une entreprise qui manipule des données client parfois très confidentielles de superviser et surveiller le réseau pour détecter toute panne, attaque ou utilisation inappropriée des ressources. Un SIEM marche donc main dans la main avec des systèmes IDS pour garantir une meilleure sécurité du système d'information. Le SIEM sert aussi de centralisateur et archiveur de logs, ce qui permet de remplir des critères ISO 27001.

Wazuh est installé sur une machine Rocky Linux 10 dans une VM contrairement à suricata qui est une machine physique.

2.3.1 Présentation de Wazuh et de son fonctionnement

Wazuh est un SIEM (Security Information and Event Management), c'est un outil qui permet de regrouper tous les événements réseau et les analyser pour ensuite détecter une menace ou un dysfonctionnement.

Pour récupérer tous ces journaux, Wazuh marche de concert avec des agents, installés sous forme de services d'arrière-plan. Ces agents récupèrent les logs de l'appareil et les transmettent au serveur Wazuh. Dans le cas où il est impossible d'installer un agent Wazuh sur l'appareil à monitorer, il est possible d'envoyer directement les journaux vers le serveur Wazuh sur une connexion UDP ou TCP, à condition de définir dans la configuration de Wazuh le format des logs et le port d'écoute.

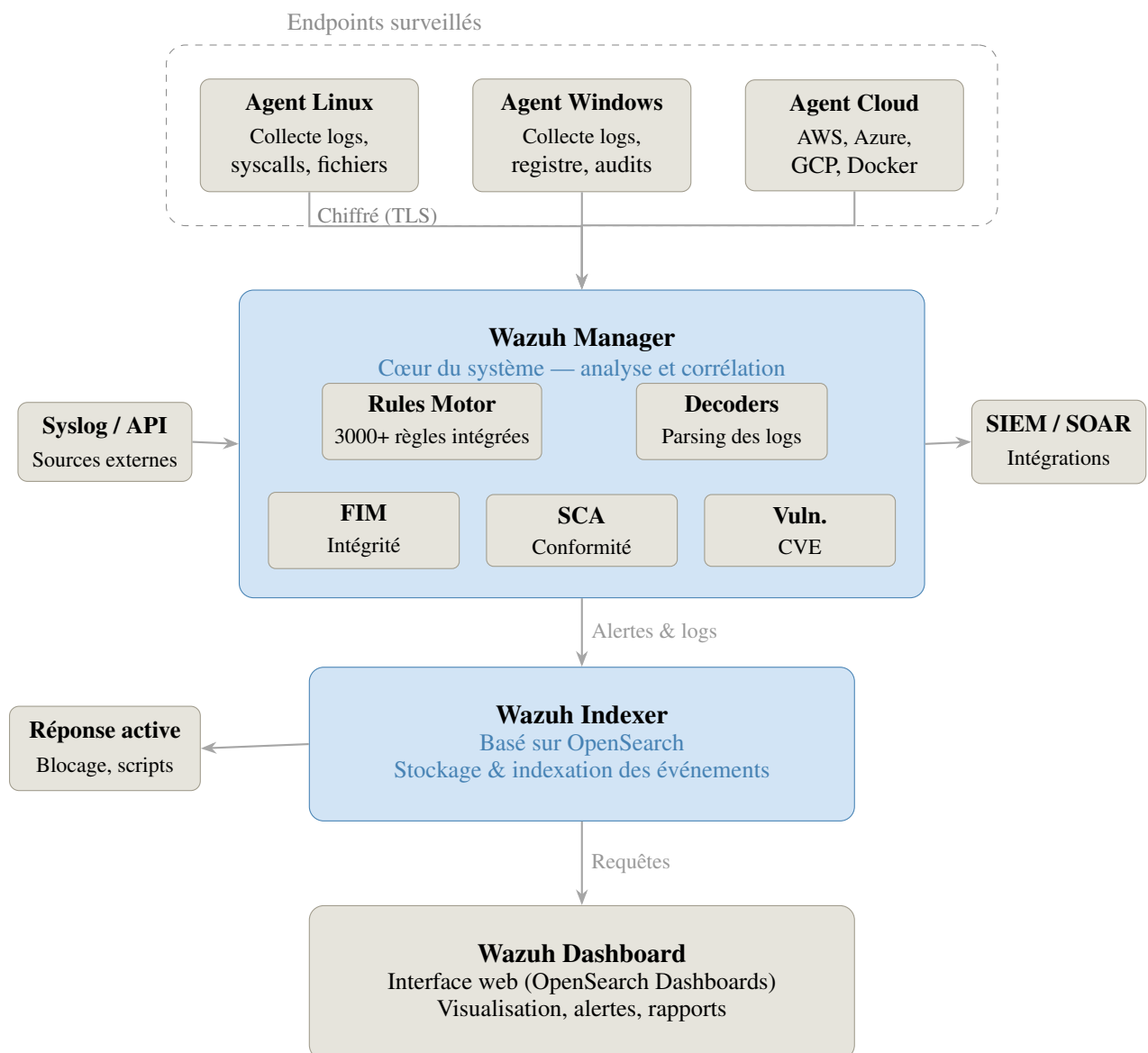


FIGURE 17 – Schéma de l'architecture de Wazuh

Le serveur Wazuh est donc constitué de 3 parties centrales, le service **wazuh-indexer** basé sur Opensearch qui indexe toutes les données pour permettre une recherche rapide au **wazuh-manager**. Le Manager est le coeur du système, c'est lui qui orchestre la collecte et l'analyse des journaux. Le service **wazuh-dashboard** permet à l'administrateur système et réseau de consulter les alertes et événements via une interface web, et de modifier la configuration de Wazuh par une interface graphique.

Chaque log reçu par Wazuh suit une chaîne de traitement définie :

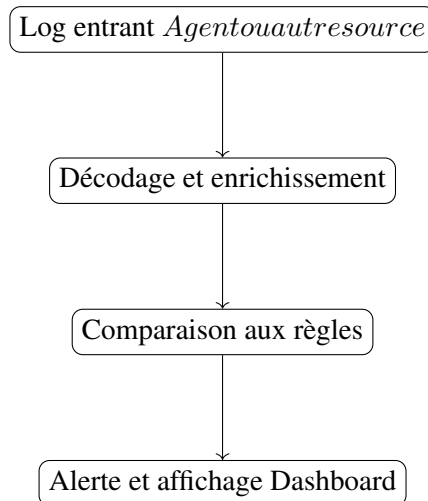


FIGURE 18 – Chaîne de traitement d'un log dans Wazuh

2.3.2 Acquisition

Avant d'analyser les logs, il faut les recevoir correctement. Ainsi nous avons des agents Wazuh présent sur l'ensemble des ordinateurs des employés, ainsi que sur les serveurs. Cependant, certains équipements nous renvoient également des logs. C'est le cas du Pare-feu Sophos et de la gateway Unify (voir [schéma de l'infrastructure](#)).

Ces deux équipements envoient leurs logs sur le port 514/tcp et 5514/tcp respectivement.

```

1  <!-- Sophos XGS107 -->
2  <remote>
3    <connection>syslog</connection>
4    <port>514</port>
5    <protocol>tcp</protocol>
6    <allowed-ips>192.168.100.1</allowed-ips>
7  </remote>
8
9  <!-- Logging du gateway Ubiquiti -->
10 <remote>
11   <connection>syslog</connection>
12   <port>5514</port>
13   <protocol>tcp</protocol>
14   <allowed-ips>192.168.105.2/32</allowed-ips>
15 </remote>
  
```

FIGURE 19 – Extrait du fichier de configuration "ossec.conf" du Manager Wazuh

SOPHOS

Rechercher

État d'intégrité du pare-feu

PROTEGER

- Règles et stratégies
- Prévention des intrusions
- Web
- Applications
- Wireless
- Email
- Serveur Web
- Réponse active aux menaces

CONFIGURER

- VPN d'accès à distance
- VPN site-à-site

Services système

Haute disponibilité | Paramètres de régulation de flux | RED | Protection antimalwares | **Paramètres du journal**

Nom *	Wazuh
Adresse IP / Domaine *	192.168.100.116
Sécuriser la transmission des journaux	☒
Port *	514
Installation *	DAEMON
Niveau de gravité *	Information
Format *	Standard syslog protocol

FIGURE 20 – Paramétrage d'envoi du journal Sophos

2.3.3 Analyse

Une fois les logs reçus par le serveur Wazuh, il faut analyser ces événements. Pour ce faire, Wazuh utilise un fonctionnement en 3 étapes :

- Premièrement, on utilise une fonction de prematch pour filtrer les logs à considérer et à ignorer.
- Ensuite, Wazuh utilise des **décodeurs**. Ces décodeurs extraient les champs importants des logs à l'aide de regex.
- Dernièrement, les champs extraits passent aux travers de **règles**. Ces règles définissent une ou plusieurs limites à ne pas dépasser, ainsi que le niveau de l'alerte à générer.

Nous allons voir ensemble un ajout de décodeurs et de règles pour détecter une connexion d'appareil non-autorisé sur le réseau Wifi.

Partons du log brut envoyé par le Gateway Unify pour une nouvelle connexion au réseau Wifi :

```
1 E.....@.?.e...i...dt.f....{^<30>Jun 23 15:21:12 UNF-Gateway UNF-
Gateway dnsmasq-dhcp[2715]: DHCPACK(br0) 192.168.106.181 52:fc:c1
:5b:51:5e Redmi-Note-11-Pro
```

FIGURE 21 – Exemple de log brut Unify - connexion établie

Les champs intéressants sont les suivants :

L'adresse MAC de l'appareil qui s'est connecté, **l'adresse IP** qui lui a été attribué et le **hostname** qui permet humain d'identifier plus facilement l'appareil.

Faisons un décodeur pour extraire ces champs :

```
1 <decoder name="ubiquiti-dhcp-ack">  
2   <prematch>dnsmaq-dhcp</prematch>  
3   <regex>DHCPACK\ ((\w+)\ )\ s+(\S+)\ s+(\S+)\ s+(\S+)\ s+(\S+)\</regex>  
4   <order>interface, srcip, srcmac, hostname</order>  
5 </decoder>
```

FIGURE 22 – Décodeur de log de connexion au réseau Wifi

Ce décodeur nommé **"ubiquiti-dhcp-ack"**, utilise le prematch **"dnsmasq-dhcp"** : Il ne considère que les logs où la chaîne de caractères **"dnsmasq-dhcp"** est présente. De la même manière, la chaîne **"DHCPACK"** est présente en texte brut afin de ne considérer seulement l'événement lorsque la connexion est établie et éviter de faux positifs.

Afin de séparer les différents champs, nous utilisons une regex, les champs à extraire sont délimités par des parenthèses. Si des parenthèses sont présentes dans le log, elles pourront être matchées en échappant la parenthèse du regex avec un antislash.

Le champ `<order>` permet de nommer les champs extraits par regex afin de les réutiliser dans les règles.

Une fois l'événement décodé, nous pouvons l'analyser à l'aide des règles suivantes :

```

1 <group name="wifi, ubiquiti, dhcp,">
2   <!-- Known device DHCPACK: level 3 (low) -->
3   <rule id="120050" level="3">
4     <decoded_as>ubiquiti-dhcp-ack</decoded_as>
5     <list field="srcmac" lookup="match_key">etc/lists/known-devices</
      list>
6     <description>Known device connected: $(hostname) (MAC: $(srcmac),
      IP: $(srcip))</description>
7   </rule>
8
9   <!-- Unknown device DHCPACK: level 12 (high) -->
10  <rule id="120051" level="12">
11    <decoded_as>ubiquiti-dhcp-ack</decoded_as>
12    <list field="srcmac" lookup="not_match_key">etc/lists/known-
      devices</list>
13    <description>ALERT! Unknown device connected: $(hostname) (MAC: $(
      srcmac), IP: $(srcip))</description>
14  </rule>
15 </group>

```

FIGURE 23 – Règles de connexion au réseau Wifi

La structure basique d'une règle wazuh nous est exposée ci-dessus. Chaque règle a un **ID** unique, une **sévérité** entre 0 et 15, et une **description** de l'alerte qui sera générée, dans laquelle nous pouvons réutiliser les champs extraits par le décodeur. Pour appliquer cette règle à la sortie du bon décodeur, on le déclare dans la balise **decoded_as**. Il nous reste le plus important à détailler, la condition d'activation.

Nous utilisons une liste **CDB**, c'est un fichier qui se présente sous forme de **clé :valeur** et qui nous permet de vérifier qu'une clé ou que l'association clé :valeur existe ou non dans ce fichier. Pour ces règles, nous avons créé une liste de toutes les adresse MAC des équipements de Mesulog, qui sont donc autorisés à se connecter sur le réseau.

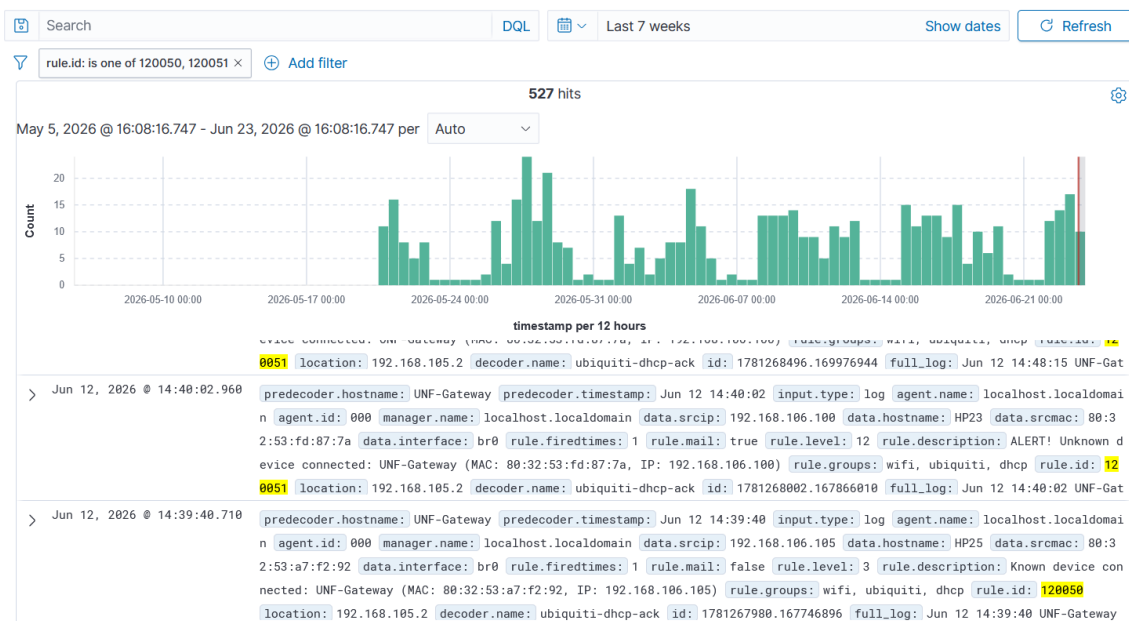


FIGURE 24 – Interface web de Wazuh regroupant les règles activées

Une fois que les règles sont déclenchées, il ne reste plus qu'à créer des alertes afin de prévenir les administrateurs système et réseau par mail.

2.3.4 Wazuh API

Wazuh possède aussi une API, accessible par le web, qui nous permet d'interagir avec le serveur Wazuh depuis d'autres applications. J'ai pu utiliser cette API pour externaliser certains traitements de logs incommodes sur Wazuh.

L'objectif principal d'analyser les journaux de logs du pare-feu Sophos était de détecter une fuite de données vers un tiers. Ainsi comme l'exemple précédent nous avons créé plusieurs décodeurs pour extraire le nombre d'octets de chaque trame enregistrée par le pare-feu. Cependant, il s'est avéré très compliqué de correctement détecter une fuite de donnée :

En effet, pour mesurer un transfert de donnée de 1Go sur 1 heure, nous pourrions faire une première règle de niveau 0 (qui ne génère pas d'alerte) avec une limite d'octets arbitraire de 10 Mo, puis une deuxième règle se déclenchant seulement si la première règle a été déclenchée 100 fois en une heure. La logique étant que 100 fois 10 Mo fait 1 Go.

Cependant, cette façon de faire (en utilisant les paramètres "frequency" et "timeframe" de Wazuh) n'est pas viable car il y a une infinité de multiplications possibles pour atteindre une limite de 1 Go. Dès lors, il est nécessaire d'externaliser ce traitement. Nous avons décidé d'utiliser un script Python pour automatiser ce processus.

Ce script nommé "sophos-exfiltration.py" est présent sur la machine Wazuh dans "/var/ossec/integrations". Étant donné que ce script ne rentre pas sur une page entière, il est disponible en [Annexe](#).

Premièrement, nous récupérons un token d'authentification à l'API Wazuh avec les identifiants d'un utilisateur créé dans Wazuh pour ce programme. Ensuite, nous recherchons tous les événements produits entre l'heure actuelle et un délai de temps défini, ici 1h avant; les arguments de cette requête sont encodés en JSON. Ensuite, nous regroupons ces données par adresse ip source et faisons la somme du nombre d'octets envoyés.

Il suffit maintenant de comparer ces sommes à la limite actuelle. Nous avons une limite de 1Gb de 8h à 19h les jours de la semaine et une limite de 100Mb la nuit(19h à 8h) et les week-ends. Ainsi si cette limite est dépassée en 1h, une alerte est générée et insérée dans le fichier d'alerte de Wazuh.

Pour que cette alerte se déclenche dans Wazuh, il faut l'ajouter dans un fichier de règle avec la forme :

```
1 <!-- API Python-->
2 <rule id="118725" level="0">
3   <field name="rule.id">118725</field>
4   <description>$(description)</description>
5   <group>firewall,data_transfer,</group>
6 </rule>
```

FIGURE 25 – Règle wazuh pour remonter l'alerte de l'intégration Python

Il est important de laisser le niveau de sévérité de cette règle à 0, car cette règle se déclenche si une alerte du même ID est active. L'alerte générée par le programme Python a déjà une sévérité qui primera sur une sévérité 0, mais la règle Wazuh pourrait se déclencher elle-même en boucle si elle émettait une alerte. Une règle au niveau 0 ne déclenche pas d'alerte, empêchant cette réaction en chaîne.

2.4 Durcissement système et administration Windows Server

L'installation de Wazuh et plus particulièrement des agents Wazuh sur les ordinateurs du parc informatique de Mesulog permet plus que de simplement remonter les journaux de logs. En effet, un agent Wazuh peut aussi vérifier en temps réel la configuration des équipements et donc la conformité à un benchmark de sécurité.

2.4.1 CIS Benchmark

Le standard de durcissement système de CIS (Center for Internet Security) est un standard de sécurité pour de nombreux systèmes d'exploitation qui s'aligne avec les exigences de la norme ISO 27001 car il permet de mesurer la gestion des accès, l'exploitation sécurisée et la protection des actifs. Tout le parc informatique (hors serveurs) est en Windows 11 entreprise. Ainsi nous nous concentrons ici sur le benchmark CIS pour Windows 11. Afin de réduire la surface d'attaques des postes, le benchmark CIS se base sur des GPOs (Group Policies Objects) à déployer sur le parc. Les benchmarks CIS sont organisés en 2 niveaux :

- le niveau 1 représente les configurations de sécurité essentielles et facilement déployables.
- le niveau 2 inclut des mesures plus restrictives adaptées aux environnements hautement sécurisés.

Le standard CIS benchmark level 1 a été retenu car il correspond le plus avec la cible (les postes employés). Ce choix a été motivé par le fait que les benchmarks CIS Level 1 sont déjà présents et intégrés dans le module Security Configuration Assessment (SCA) de Wazuh, ce qui permet la détection et la validation automatisée des configurations de sécurité sans outils supplémentaires que Wazuh.

2.4.2 Structure de l'Active Directory

Toutes les règles et recommandations du benchmark CIS ne peuvent pas être appliquées à tous les postes sans distinction. En effet certaines activités métier de l'entreprise, comme le développement ; nécessitent des permissions plus élevées que pour de la simple bureautique. On pourrait citer par exemple l'exécution de programmes non-signés.

Or l'organisation de l'Active Directory initialement présente ne permettait pas cette granularité.

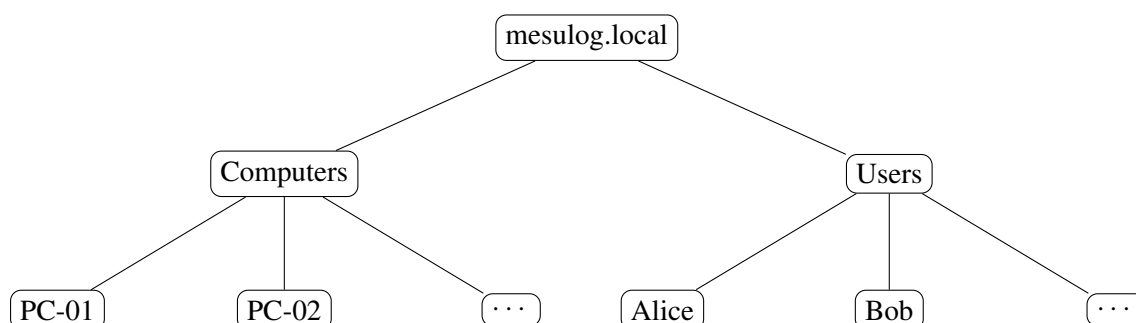


FIGURE 26 – Organisation initiale de l'Active Directory

Avant de déployer les configurations, la première étape était donc de créer un schéma structuré dans l'Active Directory (AD) afin de mieux hiérarchiser les comptes utilisateurs ainsi que les ordinateurs et serveurs présents sur le domaine. Cette structuration permet de faciliter la gestion des permissions, d'améliorer la visibilité sur les ressources et de renforcer la sécurité en limitant les accès aux ressources sensibles.

Nous avons donc remanié la structure de l'Active Directory pour y inclure des OU (Unité d'Organisation) afin de définir des règles par fonction dans l'entreprise ou par niveau de permissions accordées. Nous avons aussi regroupé des permissions d'utilisateurs dans des groupes de sécurité pour en faciliter l'administration.

Voici donc la structure de l'Active Directory après la refonte :

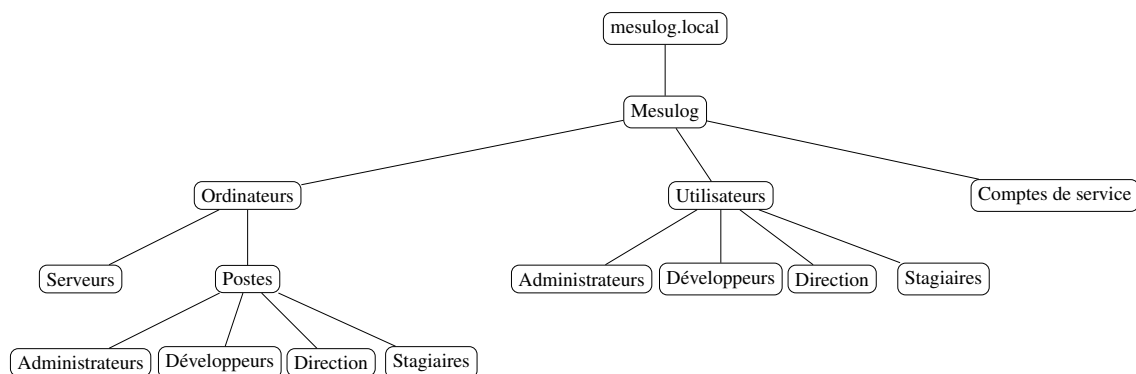


FIGURE 27 – Organisation de l'Active Directory après restructuration

Dans cette nouvelle organisation de l'annuaire Active Directory, nous pouvons notamment retrouver :

- **Ordinateurs** composé de l'OU **Postes** répartis entre les **Administrateurs**, **Développeurs**, **Direction** et **Stagiaires** ainsi que de l'OU **Serveurs**
- **Utilisateurs** composé des différents utilisateurs du domaine également répartis entre les **Administrateurs**, **Développeurs**, **Direction** et **Stagiaires**
- **Comptes de services** composé des comptes utilisés par certains serveurs et processus pour accéder à différentes ressources ou exécuter certaines fonctionnalités sur le domaine.

Ce découpage permet de mieux hiérarchiser les droits selon les personnes dans l'entreprise, les développeurs bénéficient par exemple de plusieurs droits relatifs à la virtualisation afin de développer dans des machines virtuelles, les autres fonctions de l'entreprise comme la Direction qui ne s'occupe que de documents légaux et organisationnels ne bénéficient pas de ces droits car inutiles dans leurs cas.

2.4.3 Déploiement des GPOs

Maintenant que l'Active Directory est prêt à déployer les GPOs efficacement, il faut définir le protocole de mise à niveau de conformité CIS benchmark.

Les GPOs de durcissement sont donc déployées sous forme itérative et incrémentielle. L'ensemble des GPOs du benchmark CIS ont été réparties dans un classeur Excel afin de déterminer :

- leur pertinence dans le cadre professionnel de Mesulog
- leur criticité, niveau de sécurité (faible, moyenne, élevée)
- leur impact potentiel sur les utilisateurs et les systèmes

Et donc de définir leur application ou non, leur champ d'application (Unités d'Organisations affectées) et leur priorité (dans quelle itération cette GPO est à déployer).

Le déploiement a donc été réparti en plusieurs itérations successives d'une trentaine de règles. Ces itérations sont espacées dans le temps de plusieurs jours entre chaque, afin de détecter tout problème ou dysfonctionnement par rapport à un cas d'utilisation non prévu lors de l'évaluation préliminaire.

Une convention de nommage a été établie pour faciliter la gestion et la maintenance des GPOs, par exemple, la stratégie *IT1* - Paramétrage des stratégies d'audits* est répartie selon les champs suivants :

Champ	Description
IT1	Numéro de l'itération durant laquelle la stratégie a été créée ou déployée.
*	Indique que plusieurs GPO participent à une même fonctionnalité et sont regroupées logiquement.
Paramétrage des stratégies d'audit	Description concise de l'objectif ou du contenu de la stratégie. Les termes utilisés peuvent être « Paramétrage », « Activation », « Désactivation », etc. selon la nature de la configuration appliquée.

TABLE 1 – Convention de nommage des stratégies de groupe

2.4.4 Exemples concrets de GPO appliquées

Voici des exemples de GPOs déployées sur le parc informatique :

Plan de durcissement Windows – Référentiel CIS Benchmarks Itération 2							
Basé sur les recommandations CIS Microsoft Windows – Criticité : Élevée / Moyenne / Faible Statut : À déployer / En cours / Déployé / Exclu							
N° CIS	Nom de la règle	Description (FR)	Catégorie	Criticité	Impact utilisateur	Statut	Notes de déploiement
26195	Désactiver l'accès invité SMB	Empêche les connexions anonymes aux partages.	Sécurité / Réseau	Élevée	Faible	Déployé	Compatible environnement domaine.
26193	Désactiver le protocole LLMNR	Réduit les attaques de type poisonning réseau.	Sécurité / Réseau	Élevée	Faible	Déployé	Très recommandé en entreprise.
26330	Activer BitLocker sur les postes	Chiffre les disques systèmes des utilisateurs.	Sécurité / Chiffrement	Élevée	Modéré	Déployé	Prévoir sauvegarde des clés AD.
26088	Désactiver l'impression via spooler distant	Réduit les risques liés à PrintNightmare.	Sécurité / Impression	Élevée	Faible	Déployé	/!\ A déployer seulement sur les serveur ou ordinateurs à accès restreint /!\
26116	Activer le pare-feu Windows domaine	Assure le filtrage réseau sur profil domaine.	Sécurité / Réseau	Élevée	Faible	Déployé	Déjà appliqué via GPO.

FIGURE 28 – Extrait de la 2ème itération de déploiement

2.4.5 Suivi de l'évolution de la conformité

Tous les tests de conformités sont réalisés à l'aide du module SCA de Wazuh. Nous verrons ici l'évolution de la conformité au benchmark CIS du poste "HP026" situé dans l'OU "Postes\Administrateurs" qui est le poste de travail utilisé par un des administrateurs systèmes de l'entreprise.

Pour actualiser les résultats du module SCA sur ce poste, nous pouvons soit redémarrer le poste soit utiliser la commande

```
PS C:\WINDOWS\system32> # Recuperation forcee des GPO sur le domaine
PS C:\WINDOWS\system32> gpupdate /force
Mise à jour de la stratégie...

La mise à jour de la stratégie d'ordinateur s'est terminée sans erreur.
La mise à jour de la stratégie utilisateur s'est terminée sans erreur.

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Generation de rapport sur application des GPO sur le poste HP026
PS C:\WINDOWS\system32> gpresult /H "C:\DATA\rapport.html"
```

FIGURE 29 – Commande gpupdate pour forcer la récupération des GPO sur le poste HP026

Avant application des GPOs, l'indice de conformité du poste HP026 est de :



FIGURE 30 – Conformité du poste HP026 avant la première itération de déploiement des GPO

Nous observons que la conformité est faible, de seulement 32%. La conformité moyenne des postes de travail avant le déploiement de ces GPOs se situait entre 25 et 35%.

Pour visualiser la progression obtenue au fil des itérations, le graphique ci-dessous présente l'évolution du taux de conformité aux standards CIS Level 1 pour le poste HP026, depuis l'état initial jusqu'à la quatrième itération.

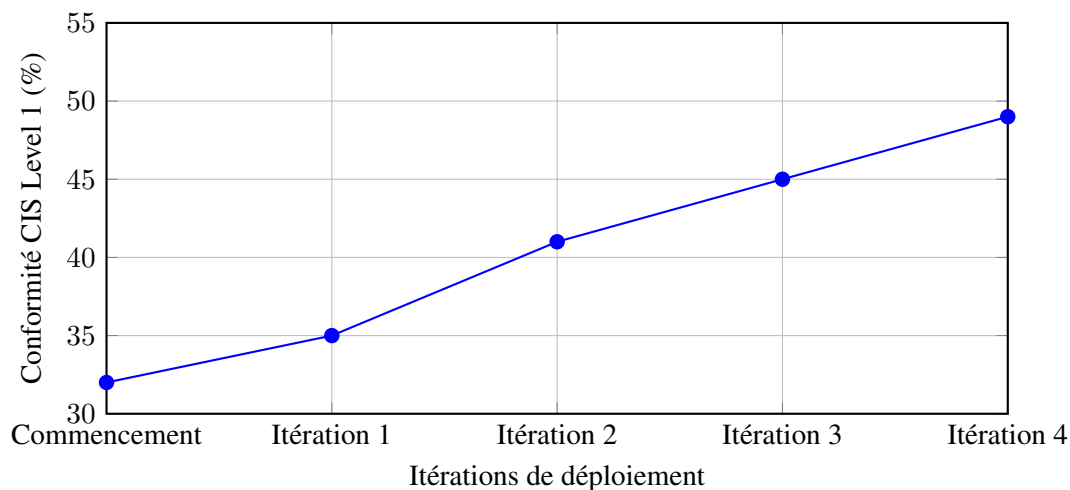


FIGURE 31 – Évolution de la conformité CIS Level 1 du poste HP026 sur 4 itérations

A l'issue des 4 premières itérations, le taux de conformité est passé à 49%, ce qui représente une amélioration significative par rapport à l'état initial de 32%. Il reste cependant une marge de progression importante avant d'atteindre un objectif minimal de 70% de conformité.

2.4.6 Limitation du module SCA de Wazuh

Au cours de notre utilisation du module SCA de Wazuh, nous nous sommes aperçus de certaines limitations :

Pour tester la conformité du poste au benchmark CIS, l'agent Wazuh présent sur celui-ci récupère un fichier YAML depuis le template présent sur le serveur Wazuh, puis exécute les commandes présentes dans ce fichier, avant de renvoyer leurs résultats au serveur Wazuh.

Cependant, nous avons constaté après l'application de plusieurs GPO que le taux de conformité n'augmentait pas malgré le fait que les GPO soient appliquées sur les machines. Nous avons donc vérifié en générant un rapport des GPOs appliquées sur le poste que les GPOs étaient bel et bien actives :

Computer Configuration (Enabled)	
Policies	
Windows Settings	
Security Settings	
Local Policies/User Rights Assignment	
Local Policies/Security Options	
Devices	
Policy	Setting
Devices: Restrict CD-ROM access to locally logged-on user only	Disabled
Devices: Restrict floppy access to locally logged-on user only	Disabled
User Account Control	
Policy	Setting
User Account Control: Behavior of the elevation prompt for administrators in Admin Approval Mode	Elevate without prompting
Registry Values	
Policy	Setting
MACHINE\Software\Microsoft\Driver Signing\Policy	1
MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer\NoDriveTypeAutoRun	255
MACHINE\System\CurrentControlSet\Control\Lsa\FIPSAlgorithmPolicy	1
MACHINE\System\CurrentControlSet\Services\Tcpip\Parameters\SynAttackProtect	1
MACHINE\System\CurrentControlSet\Services\Tcpip\Parameters\TCPMaxPortsExhausted	5

FIGURE 32 – Extrait d'un rapport d'application des GPO sur le poste HP026

Nous avons donc vérifié directement les commandes utilisées par l'agent Wazuh pour tester la conformité :

```
PS C:\WINDOWS\system32> # Commande pour vérifier que la GPO "Audit Credential Validation is set to Success and Failure" est appliquée
PS C:\WINDOWS\system32> auditpol.exe /get /subcategory:"Credential Validation"
L'erreur 0x00000057 s'est produite :
Paramètre incorrect.

Utilisation : Commande AuditPol [<sous-commande><options>]

Commandes (une seule commande autorisée par exécution)
/? Aide (contextuelle)
/get Affiche la stratégie d'audit actuelle.
/set Définit la stratégie d'audit.
/list Affiche les éléments de stratégie sélectionnables.
/backup Enregistre une stratégie d'audit dans un fichier.
/restore Restaure une stratégie d'audit à partir d'un fichier.
/clear Efface une stratégie d'audit.
/remove Supprime une stratégie d'audit par utilisateur pour un compte d'utilisateur.
/resourceSACL Configure les listes SACL de ressource globale.

Utilisez AuditPol <commande> /? pour obtenir des détails sur chaque commande
```

FIGURE 33 – Test manuel d'une commande utilisée par l'agent, sur le poste HP026

En ouvrant le fichier *YAML* envoyé par Wazuh sur les machines nous avons constaté que quand le module SCA testait l'application d'une GPO, il vérifiait la configuration d'une clé de registre spécifique. Pour les GPO qui n'étaient pas liées à des clés de registre, le module teste directement la conformité d'un paramètre via une commande PowerShell. Cependant, ces commandes sont écrites pour un système d'exploitation en anglais, et les postes de Mesulog sont tous installés en français. Dans notre cas, le nom de la GPO était "Validation des informations d'identification" au lieu de "Credential Validation".

Pour pallier ce problème, nous avons modifié les commandes utilisées par le module pour utiliser le GUID à la place du nom anglais et que les résultats attendus soient en français. Nous avons donc été obligés de modifier tout ce fichier pour traduire chaque instance de test de GPO utilisant la commande **"auditpol"**.

Ce changement dans le fichier de test a engendré un autre problème. En effet, l'agent Wazuh exécute les commandes via PowerShell, ce qui limite le support des caractères UTF-8. Nous avons donc entouré la commande **"auditpol"** d'une autre commande PowerShell spécifiant l'encodage UTF-8, pour ainsi utiliser directement les caractères accentués comme "Succès et échec". Nous pouvons voir ici un exemple de commande de test de ces GPO mise à jour :

```
PS C:\WINDOWS\system32> "c:\powershell -Command \"auditpol /get /subcategory:'{0CCE923F-69AE-11D9-BED3-505054503030}' -match 'Succès et échec' { echo 'PASS' } else { echo 'FAIL' }\" -> r:PASS"
```

FIGURE 34 – Commande PowerShell modifiée pour supporter les accents

Il ne suffit donc maintenant plus qu'à déployer automatiquement le nouveau fichier de test sur le parc informatique. Nous pouvons simplement utiliser une GPO pour modifier ce fichier sur les postes. Ce fichier sera donc disponible dans le volume partagé **SYSVOL**.

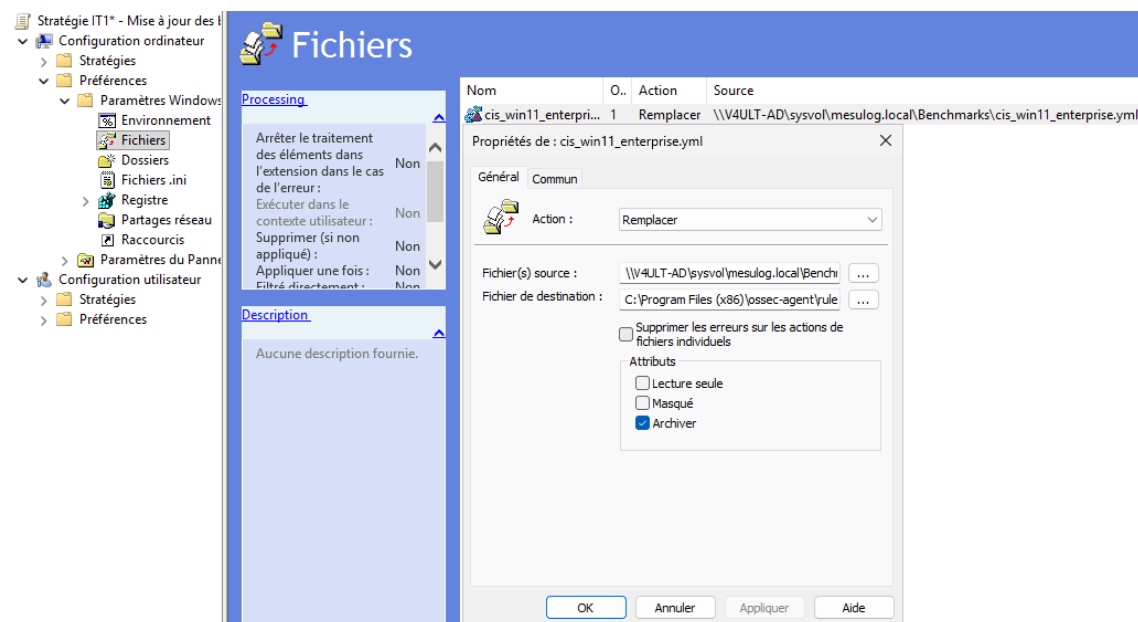


FIGURE 35 – Extrait de la GPO permettant de déployer le fichier de test modifié sur les postes

3 Impact environnemental et sociétal

3.1 Déplacements

Dans le cadre de mon stage, j'ai fait des aller-retour de Voiron à Moirans tous les jours. Mon mode de déplacement est conditionnel de la météo. En effet lorsqu'il n'est pas prévu de pleuvoir, je fais mes trajets de 8.4km aller et 8.4km retour en vélo mécanique, ce qui émet naturellement 0g de CO2 par jour.

Au contraire, lorsqu'il menace de pleuvoir je me rends au travail en voiture essence, ce qui émet 170 gCO2e/km ou 2.856 kgCO2e par journée.

L'utilisation de la voiture a donc un impact certain mais qui reste limité car peu fréquent. Ces estimations ont été faites avec le simulateur en ligne de l'ADEME.

3.2 Répercussions écologiques et sociales des actions menées par l'entreprise

NERYS GROUP a intégré dans son Système de Management de la Qualité un volet management environnemental (§ 1.6 du MAQ-07-07-033). En fonction des besoins spécifiques de nos clients, cette partie management environnemental peut être complétée par un chapitre dédié dans le Plan de Management Qualité du projet concerné, telles que la gestion d'éliminations de déchets ou la gestion des aspects de pollutions. Consciente de son rôle dans la préservation de l'environnement, NERYS GROUP veille à ce que l'ensemble de ses activités n'impacte pas négativement ses parties intéressées ni son écosystème. Cette vigilance environnementale est une composante essentielle de notre responsabilité sociétale.

4 Conclusion

Au cours de ce stage chez Mesulog, j'ai pu rencontrer des sujets variés et formateurs, comme le traitement et l'analyse de données, la mise en place d'un système de supervision et de sécurisation du réseau informatique d'une entreprise.

L'installation et la configuration de Wazuh en tant que solution SIEM a permis d'établir un système complet de collecte et d'analyse des événements réseau, avec la création de décodeurs, de règles et d'alertes personnalisées ; ce qui m'a permis d'explorer en profondeur les possibilités de Wazuh. Notamment avec l'intégration d'un script externe à l'API pour l'analyse des événements du pare-feu Sophos.

La refonte de l'Active Directory et le déploiement progressif des stratégies relatives au benchmarks CIS Level 1 via des GPO ont grandement renforcé la sécurité du parc informatique. Bien que le score de conformité n'ait pas encore atteint l'objectif affiché, le passage de 32% à 49% de conformité pour le poste HP026 prouve son efficacité et son potentiel.

Ce stage m'a aussi permis de développer des compétences professionnelles importantes comme la maîtrise des solutions SIEM, une compréhension approfondie de la gestion de Windows Server et d'Active Directory, la capacité à identifier des problèmes et la capacité à utiliser des moyens alternatifs (installation de Suricata dans un conteneur, externalisation de la mise à jour des règles d'IP malicieuses) pour assurer le bon fonctionnement des services.

Plusieurs pistes d'amélioration existent cependant. De manière évidente une poursuite des itérations de mise en conformité avec le benchmark CIS Windows 11, mais aussi avec l'ajout de nombreuses autres règles pour Wazuh afin d'éliminer les dernières zones d'ombre du parc informatique de Mesulog.

Dernièrement, ce stage m'a confirmé mon orientation vers le domaine de la cybersécurité et de l'administration systèmes & réseaux. J'ai acquis une expérience concrète qu'il me tarde de réutiliser dans mes emplois futurs.

Je tiens à remercier sincèrement l'équipe de Mesulog pour l'accueil, les conseils, l'aide et les opportunités d'apprentissage qu'ils m'ont offertes au cours de ce stage.

5 Annexe

Requêtes de nettoyages

```
1  --SETUP-01-ModelUpdateEmpty
2
3  UPDATE RAW_DATA
4  SET
5      Model = DLookup (
6          "[Model]",
7          "RAW_DATA",
8          "UHF100name='" & [UHF100name] & "'" AND [Model] Is Not Null AND
9              Trim([Model] & '') <> ''"
10     )
11 WHERE
12     [Model] IS NULL
13     OR Trim([Model] & '') = '';
14
15 --SETUP-02-ModelUpdateUHF100-V2
16
17 UPDATE RAW_DATA
18 SET
19     Model = "UHF100 V2"
20 WHERE
21     [Model] = "UHF100 2 GHz";
22
23 --SETUP-03-ModelNormalise-1
24
25 UPDATE RAW_DATA
26 SET
27     ModelNormalise = IIf(
28         Model = "UHF100",
29         1,
30         IIf(Model = "UHF100 V2", 2, 0)
31     );
32
33 --SETUP-04-ModelNormalise-2
34
35 SELECT DISTINCT
36     SerialNumberNormalise
37 INTO
38     temp_model1_serials
39 FROM
40     RAW_DATA
41 WHERE
42     ModelNormalise = 1;
43
44 --SETUP-05-ModelNormalise-3
45
46 SELECT DISTINCT
47     SerialNumberNormalise
48 INTO
49     temp_model2_serials
50 FROM
51     RAW_DATA
52 WHERE
53     ModelNormalise = 2;
```

```

1  --SETUP-06-ModelNormalise-4
2
3  UPDATE RAW_DATA
4  SET
5      ModelNormalise = IIf(
6          Year(StartTime) >= 2025,
7          2,
8          IIf(
9              Year(StartTime) <= 2020,
10             1,
11             IIf(
12                 DCount (
13                     "*",
14                     "temp_model1_serials",
15                     "SerialNumberNormalise = '" &
16                     SerialNumberNormalise & "'"
17                 ) > 0,
18                 1,
19                 IIf(
20                     DCount (
21                         "*",
22                         "temp_model2_serials",
23                         "SerialNumberNormalise = '" &
24                         SerialNumberNormalise & "'"
25                     ) > 0,
26                     2,
27                     ModelNormalise
28                 )
29             )
30         )
31 WHERE
32     ModelNormalise = 0
33     OR ModelNormalise IS NULL;
34
35 --SETUP-07-SerialNumberNorlalise-1
36
37 UPDATE RAW_DATA
38 SET
39     SerialNumber = IIf(
40         Left(SerialNumber, 1) = " ",
41         Trim(SerialNumber),
42         IIf(
43             Left(Trim(SerialNumber), 3) = "BF "
44             OR Left(Trim(SerialNumber), 3) = "BF:",
45             "BF-" & Mid (Trim(SerialNumber), 4),
46             IIf(
47                 Left(Trim(SerialNumber), 5) = "BF : ",
48                 "BF-" & Mid (Trim(SerialNumber), 6),
49                 SerialNumber
50             )
51         )
52     );

```

FIGURE 37 – Requêtes de nettoyage des données -2ème partie

```

1  --SETUP-08-SerialNumberNormalise-2
2
3  UPDATE RAW_DATA
4  SET
5      SerialNumberNormalise = UCase (
6          IIf(
7              InStr (SerialNumber, " ") > 0,
8              Left (SerialNumber, InStr (SerialNumber, " ") -1),
9              IIf(
10                 InStr (SerialNumber, "_") > 0,
11                 Left (SerialNumber, InStr (SerialNumber, "_") -1),
12                 SerialNumber
13             )
14         )
15     );
16
17  --SETUP-09-SerialNumberNormalise-3
18
19  UPDATE RAW_DATA
20  SET
21      SerialNumberNormalise = IIf(
22          InStr (SerialNumberNormalise, ".") > 0,
23          Left (
24              SerialNumberNormalise,
25              InStr (SerialNumberNormalise, ".") -1
26          ) & "." & Right (
27              "000" & Mid (
28                  SerialNumberNormalise,
29                  InStr (SerialNumberNormalise, ".") + 1
30              ),
31              3
32          ),
33          SerialNumberNormalise
34      )
35  WHERE
36      InStr (SerialNumberNormalise, ".") > 0;
37
38  --SETUP-10-Prod_HorsProd-1
39
40  UPDATE RAW_DATA
41  SET
42      Prod = "HORS_PROD"
43  WHERE
44      SerialNumber LIKE '*SAV*'
45      OR SerialNumber LIKE '*Sav*'
46      OR SerialNumber LIKE '*sav*'
47      OR SerialNumber LIKE '*20min*'
48      OR SerialNumber LIKE '*20 min*'
49      OR SerialNumber LIKE '*maj v4.10*';

```

FIGURE 38 – Requêtes de nettoyage des données -3ème partie


```

1  --SETUP-11-Prod_HorsProd-2
2
3  UPDATE RAW_DATA
4  SET
5      Prod = "PROD"
6  WHERE
7      NOT Prod = "HORS_PROD";
8
9  --SETUP-12-RAW_DATAReduit
10
11 SELECT
12     StartTime,
13     StationID,
14     TotalTime,
15     StepCount,
16     UUTresult,
17     ModelNormalise,
18     SerialNumberNormalise,
19     Prod
20 INTO
21     RAW_DATA_REDUIT
22 FROM
23     RAW_DATA
24 WHERE
25     Year(StartTime) >= 2020
26     AND StepCount <= 39
27 ORDER BY
28     StartTime;

```

FIGURE 39 – Requêtes de nettoyage des données -4ème partie

Requêtes d'extractions

```
1  --BC-MOD-00-TEST_DATAProd
2  SELECT
3      *
4  INTO
5      TEST_DATA_PROD_B1
6  FROM
7      TEST_DATA_REDUIT
8  WHERE
9      Prod = "PROD"
10     AND StationID = "VM_T-D14"
11     AND ModelNormalise = 2;
12
13  --BC-MOD-01-TableEssaisStatut
14  SELECT
15      StartTime,
16      SerialNumberNormalise,
17      ModelNormalise,
18      TotalTime,
19      IIf(UUTresult = "Passed", True, False) AS Passed,
20      IIf(UUTresult = "Failed", True, False) AS Failed,
21      IIf(UUTresult = "Terminated", True, False) AS Terminated,
22      IIf(UUTresult = "Error", True, False) AS Error
23  INTO
24      B1_table_essais_statut
25  FROM
26      TEST_DATA_PROD_B1
27  ORDER BY
28      StartTime;
29
30  --BC-MOD-02-TableFirstPass
31  SELECT
32      SerialNumberNormalise,
33      Min(StartTime) AS FirstPassTime
34  INTO
35      B1_table_first_pass
36  FROM
37      B1_table_essais_statut
38  WHERE
39      Passed = True
40  GROUP BY
41      SerialNumberNormalise;
42
43  --BC-MOD-03-TempLastIsError
44  SELECT
45      SerialNumberNormalise,
46      Max(StartTime) AS LastTime,
47      Max(IIf(Error = True, 1, 0)) AS LastIsError
48  INTO
49      B1_temp_LastIsError
50  FROM
51      B1_table_essais_statut AS bht1
52  WHERE
53      NOT EXISTS (
54          SELECT
55              1
56          FROM
57              B1_table_first_pass AS bht2
58          WHERE
59              bht2.SerialNumberNormalise = bht1.SerialNumberNormalise
60      )
61  GROUP BY
62      SerialNumberNormalise;
```

```

1  --BC-MOD-04-TableAttemptCount
2  SELECT
3      n.SerialNumberNormalise,
4      IIf(
5          EXISTS (
6              SELECT
7                  1
8              FROM
9                  B1_table_first_pass
10             WHERE
11                 SerialNumberNormalise = n.SerialNumberNormalise
12         ),
13         (
14             SELECT
15                 Count (*)
16             FROM
17                 B1_table_essais_statut AS sub
18             WHERE
19                 sub.SerialNumberNormalise = n.SerialNumberNormalise
20                 AND sub.StartTime <= (
21                     SELECT
22                         FirstPassTime
23                     FROM
24                         B1_table_first_pass
25                     WHERE
26                         SerialNumberNormalise = n.
27                             SerialNumberNormalise
28                 )
29         ),
30         IIf(
31             EXISTS (
32                 SELECT
33                     1
34                 FROM
35                     B1_temp_LastIsError
36                 WHERE
37                     SerialNumberNormalise = n.SerialNumberNormalise
38                     AND LastIsError = 1
39             ),
40             -2,
41             -1
42         ) AS AttemptCount,
43         (
44             SELECT
45                 Max(StartTime)
46             FROM
47                 B1_table_essais_statut AS t1
48             WHERE
49                 t1.SerialNumberNormalise = n.SerialNumberNormalise
50         ) AS EndDate,
51         ModelNormalise
52 INTO
53     B1_table_AttemptCount
54 FROM
55     (
56         SELECT DISTINCT
57             SerialNumberNormalise,
58             ModelNormalise
59         FROM
60             B1_table_essais_statut
61     ) AS n;

```

FIGURE 41 – Requêtes d'extraction de données -2eme partie

```

1  --BC-MOD-05-TableEssaisStatutTotal
2  SELECT
3      Sum(IIf(Passed = True, 1, 0)) AS Essai_Passed,
4      Sum(IIf(Failed = True, 1, 0)) AS Essai_Failed,
5      Sum(IIf(Terminated = True, 1, 0)) AS Essai_Terminated,
6      Sum(IIf(Error = True, 1, 0)) AS Essai_Error,
7      Count(*) AS Total,
8      Year(StartTime) AS Annee
9  INTO
10     B1_table_essais_statut_total
11  FROM
12     B1_table_essais_statut
13  GROUP BY
14     Year(StartTime);
15
16  --BC-MOD-06-TempNbProd_HorsProd
17  SELECT
18      Year(Max(StartTime)) AS Annee,
19      SerialNumberNormalise
20  INTO
21     B1_temp_Nb_HorsProd
22  FROM
23     TEST_DATA_REDUIT
24  WHERE
25     StationID = "VM_T-D14"
26     AND Prod = "HORS_PROD"
27     AND ModelNormalise = 2
28  GROUP BY
29     SerialNumberNormalise;
30
31  --BC-MOD-07-TableNbModules
32  SELECT
33      Year(t1.EndDate) AS Annee,
34      (
35          SELECT
36              COUNT(*)
37          FROM
38              B1_table_AttemptCount AS t2
39          WHERE
40              t2.AttemptCount > 0
41              AND Year(t2.EndDate) = Year(t1.EndDate)
42      ) AS Valides,
43      COUNT(*) AS TesteProd,
44      (
45          (
46              SELECT
47                  count(*)
48              FROM
49                  B1_temp_Nb_HorsProd AS b2
50              WHERE
51                  b2.Annee = Year(t1.EndDate)
52          ) + Count(*)
53      ) AS TesteTotal
54  INTO
55     B1_Nb_Module_Prod
56  FROM
57     B1_table_AttemptCount AS t1
58  GROUP BY
59     Year(t1.EndDate);

```

FIGURE 42 – Requêtes d'extraction de données -3eme partie

```

1  --BC-MOD-08-TableEffortPassMoyenne
2  SELECT
3      Avg(AttemptCount) AS Moyenne,
4      Year(EndDate) AS Annee
5  INTO
6      B1_table_EffortPass_moyenne
7  FROM
8      B1_table_AttemptCount
9  WHERE
10     AttemptCount > 0
11  GROUP BY
12     Year(EndDate);
13
14  --BC-MOD-09-TempSortedAttemptCount
15  SELECT DISTINCT
16     SerialNumberNormalise,
17     AttemptCount,
18     Year(EndDate) AS Annee
19  INTO
20     B1_temp_sorted_AttemptCount
21  FROM
22     B1_table_AttemptCount
23  WHERE
24     AttemptCount > 0
25  ORDER BY
26     Year(EndDate),
27     AttemptCount;

```

FIGURE 43 – Requêtes d'extraction de données -4eme partie

```

1  --BC-MOD-10-TableEffortPassMediane
2  SELECT
3      TOP 1 (
4          SELECT
5              Max(t1bis.AttemptCount)
6          FROM
7              (
8                  SELECT
9                      TOP 50 PERCENT t1.AttemptCount
10                 FROM
11                     B1_temp_sorted_AttempCount AS t1
12                 WHERE
13                     t1.Annee = 2020
14             ) AS t1bis
15      ) AS med2020,
16      (
17          SELECT
18              Max(t2bis.AttemptCount)
19          FROM
20              (
21                  SELECT
22                      TOP 50 PERCENT t2.AttemptCount
23                 FROM
24                     B1_temp_sorted_AttempCount AS t2
25                 WHERE
26                     t2.Annee = 2021
27             ) AS t2bis
28      ) AS med2021,
29      (
30          SELECT
31              Max(t3bis.AttemptCount)
32          FROM
33              (
34                  SELECT
35                      TOP 50 PERCENT t3.AttemptCount
36                 FROM
37                     B1_temp_sorted_AttempCount AS t3
38                 WHERE
39                     t3.Annee = 2022
40             ) AS t3bis
41      ) AS med2022,
42      (
43          SELECT
44              Max(t4bis.AttemptCount)
45          FROM
46              (
47                  SELECT
48                      TOP 50 PERCENT t4.AttemptCount
49                 FROM
50                     B1_temp_sorted_AttempCount AS t4
51                 WHERE
52                     t4.Annee = 2023
53             ) AS t4bis
54      ) AS med2023,

```

FIGURE 44 – Requêtes d'extraction de données -5eme partie

```

1      (
2          SELECT
3              Max(t5bis.AttemptCount)
4          FROM
5              (
6                  SELECT
7                      TOP 50 PERCENT t5.AttemptCount
8                  FROM
9                      B1_temp_sorted_AttempCount AS t5
10                 WHERE
11                     t5.Annee = 2024
12             ) AS t5bis
13      ) AS med2024,
14      (
15          SELECT
16              Max(t6bis.AttemptCount)
17          FROM
18              (
19                  SELECT
20                      TOP 50 PERCENT t6.AttemptCount
21                  FROM
22                      B1_temp_sorted_AttempCount AS t6
23                 WHERE
24                     t6.Annee = 2025
25             ) AS t6bis
26      ) AS med2025,
27      (
28          SELECT
29              Max(t7bis.AttemptCount)
30          FROM
31              (
32                  SELECT
33                      TOP 50 PERCENT t7.AttemptCount
34                  FROM
35                      B1_temp_sorted_AttempCount AS t7
36                 WHERE
37                     t7.Annee = 2026
38             ) AS t7bis
39      ) AS med2026
40 INTO
41     B1_table_EffortPass_mediane
42 FROM
43     B1_temp_sorted_AttempCount AS t8;

```

FIGURE 45 – Requêtes d'extraction de données -6eme partie

```

1  --BC-MOD-11-TempTimestampEssais
2  SELECT
3      Hour (StartTime) AS HeureDeDebut,
4      Weekday (StartTime) AS Jour,
5      Year(StartTime) AS Annee,
6      Sum(IIf(UUTresult = "Passed", 1, 0)) AS Essai_Passed,
7      Sum(IIf(UUTresult = "Failed", 1, 0)) AS Essai_Failed,
8      Sum(IIf(UUTresult = "Terminated", 1, 0)) AS Essai_Terminated,
9      Sum(IIf(UUTresult = "Error", 1, 0)) AS Essai_Error,
10     count(*) AS Total
11 INTO
12     B1_temp_timestamp_essais
13 FROM
14     TEST_DATA_PROD_B1
15 GROUP BY
16     Year(StartTime),
17     Weekday (StartTime),
18     Hour (StartTime);
19
20 --BC-MOD-12-TableErrorLundi
21 SELECT
22     Sum(t1.Essai_Error) AS Nb_error,
23     (
24         SELECT
25             (Sum(t2.Essai_Error) / Sum(t2.Total))
26         FROM
27             B1_temp_timestamp_essais AS t2
28         WHERE
29             t2.Jour = 2
30             AND t2.HeureDeDebut BETWEEN 6 AND 10
31             AND t1.Annee = t2.Annee
32     ) AS Ratio_error,
33     t1.Annee
34 INTO
35     B1_table_error_lundi
36 FROM
37     B1_temp_timestamp_essais AS t1
38 WHERE
39     Jour = 2
40     AND HeureDeDebut BETWEEN 6 AND 10
41 GROUP BY
42     Annee;

```

FIGURE 46 – Requêtes d'extraction de données -6eme partie


```

1  --BC-MOD-13-TableErrorMMJV
2  SELECT
3      (Sum(t1.Essai_Error) / 4) AS Nb_error,
4      (
5          SELECT
6              (Sum(t2.Essai_Error) / Sum(t2.Total))
7          FROM
8              B1_temp_timestamp_essais AS t2
9          WHERE
10             t2.Jour BETWEEN 3 AND 6
11             AND t2.HeureDeDebut BETWEEN 6 AND 10
12             AND t1.Annee = t2.Annee
13      ) AS Ratio_error,
14      t1.Annee
15  INTO
16      B1_table_error_MMJV
17  FROM
18      B1_temp_timestamp_essais AS t1
19  WHERE
20      Jour BETWEEN 3 AND 6
21      AND HeureDeDebut BETWEEN 6 AND 10
22  GROUP BY
23      Année;
24
25  --BC-MOD-14-TableRatioV1V2
26  SELECT
27      Sum(IIf(ModelNormalise = 1, 1, 0)) AS V1,
28      Sum(IIf(ModelNormalise = 2, 1, 0)) AS V2,
29      (
30          (
31              (Sum(IIf(ModelNormalise = 2, 1, 0))) / (Count(*) / 100)
32          )
33      ) AS Ratio_V2_Total,
34      Year(EndDate) AS Année
35  INTO
36      B1_table_ratio_v1v2
37  FROM
38      B1_table_AttemptCount
39  GROUP BY
40      Year(EndDate);

```

FIGURE 47 – Requêtes d'extraction de données -7eme partie

```

1  --BC-MOD-15-TempTimediff
2  SELECT
3      SerialNumberNormalise,
4      Min(StartTime) AS FirstStart,
5      Min(IIf(Passed = True, StartTime, NULL)) AS FirstPass,
6      IIf(
7          Min(IIf(Passed = True, StartTime, NULL)) IS NULL,
8          NULL,
9          DateDiff(
10             "s",
11             Min(StartTime),
12             Min(IIf(Passed = True, StartTime, NULL))
13         )
14     ) AS DiffSeconds,
15     Min(IIf(Passed = True, TotalTime, NULL)) AS TestTimeSeconds,
16     Round(
17         (
18             (
19                 IIf(
20                     Min(IIf(Passed = True, StartTime, NULL)) IS NULL,
21                     NULL,
22                     DateDiff(
23                         "s",
24                         Min(StartTime),
25                         Min(IIf(Passed = True, StartTime, NULL))
26                     )
27                 ) + Min(IIf(Passed = True, TotalTime, NULL))
28             ) / 3600
29         ),
30         3
31     ) AS TimeToFirstPassHours
32 INTO
33     B1_temp_timediff
34 FROM
35     B1_table_essais_statut
36 GROUP BY
37     SerialNumberNormalise;
38
39 --BC-MOD-16-TempSortedTimediff
40 SELECT DISTINCT
41     SerialNumberNormalise,
42     TimeToFirstPassHours,
43     Year(FirstPass) AS Annee
44 INTO
45     B1_temp_sorted_TimeDiff
46 FROM
47     B1_temp_timediff
48 WHERE
49     TimeToFirstPassHours IS NOT NULL
50 ORDER BY
51     Year(FirstPass),
52     TimeToFirstPassHours;

```

FIGURE 48 – Requêtes d'extraction de données -8eme partie

```

1
2  --BC-MOD-17-TableTtFP
3  SELECT
4      Year(FirstPass) AS Annee,
5      Avg(TimeToFirstPassHours) AS TTFP_moyen,
6      DMedian (
7          "TimeToFirstPassHours",
8          "B1_temp_timediff",
9          "FirstPass IS NOT NULL AND Year(FirstPass) = " & Year(
10         FirstPass)
11     ) AS TTFP_median
12 INTO
13     B1_TTFP
14 FROM
15     B1_temp_timediff
16 WHERE
17     FirstPass IS NOT NULL
18 GROUP BY
19     Year(FirstPass);

```

FIGURE 49 – Requêtes d'extraction de données -9eme partie

fichier sophos-exfiltration.py

```
1  #!/usr/bin/env python3
2  import json
3  import requests
4  import sys
5  from datetime import datetime, timedelta
6  from collections import defaultdict
7
8  # Configuration
9  WAZUH_API = "https://localhost:55000"
10 WAZUH_USER = "#####"
11 WAZUH_PASS = "#####"
12 TIME_WINDOW_MINUTES = 60
13 ALERT_FILE = "/var/ossec/logs/alerts/alerts.json"
14 DAY_THRESHOLD_OCTETS = 1_000_000_000 # 1GB limite du jour
15 NIGHT_THRESHOLD_OCTETS = 100_000_000 # 100MB limite de nuit/weekend
16 DAY_START_HOUR = 8
17 DAY_END_HOUR = 19
18
19 # Recuperer un token d'authentification
20 def get_token():
21     r = requests.post(f"{WAZUH_API}/security/user/authenticate",
22                       auth=(WAZUH_USER, WAZUH_PASS), verify=False)
23     return r.json()["data"]["token"]
24
25 # retourne la limite en vigueur
26 def get_current_threshold():
27     now = datetime.now()
28     current_hour = now.hour
29     current_weekday = now.weekday() # 0=lundi, 6=dimanche
30
31     # Weekend
32     if current_weekday >= 5:
33         return NIGHT_THRESHOLD_OCTETS, "weekend"
34
35     # nuit semaine
36     if current_hour < DAY_START_HOUR or current_hour >= DAY_END_HOUR:
37         return NIGHT_THRESHOLD_OCTETS, "night"
38
39     # jour semaine
40     return DAY_THRESHOLD_OCTETS, "daytime"
```

FIGURE 50 – Script sophos-exfiltration.py -1ere partie

```

1  #recuperer les evenements depuis 1h
2  def get_events(token):
3      now = datetime.utcnow()
4      one_hour_ago = now - timedelta(minutes=TIME_WINDOW_MINUTES)
5
6      query = {
7          "query": {
8              "bool": {
9                  "must": [
10                     {"term": {"rule.groups": "sophos"}},
11                     {"term": {"decoder.name": "sophos-firewall"}},
12                     {"range": {"timestamp": {
13                         "gte": one_hour_ago.isoformat(),
14                         "lte": now.isoformat()
15                     }}}
16                 ]
17             }
18         }
19     }
20
21     headers = {"Authorization": f"Bearer {token}"}
22     r = requests.post(f"{WAZUH_API}/events", json=query, headers=
23         headers, verify=False)
24     return r.json()
25
26 # somme des octets par adresse IP (ip source)
27 def analyze_exfiltration(events):
28     ip_totals = defaultdict(lambda: {"total_octets": 0, "connections":
29         0, "destinations": set()})
30
31     for event in events:
32         src_ip = event.get("data", {}).get("src_ip", "unknown")
33         octets_sent = int(event.get("data", {}).get("bytes_sent", 0))
34         dst_ip = event.get("data", {}).get("dst_ip", "unknown")
35
36         ip_totals[src_ip]["total_octets"] += octets_sent
37         ip_totals[src_ip]["connections"] += 1
38         ip_totals[src_ip]["destinations"].add(dst_ip)
39
40     return ip_totals

```

FIGURE 51 – Script sophos-exfiltration.py -2ème partie

```

1  #crer une alerte
2  def generate_alert(src_ip, data, threshold_octets, period_name):
3      total_mb = data['total_octets'] / 1_000_000
4      threshold_mb = threshold_octets / 1_000_000
5
6      if period_name == "night":
7          severity = "HIGH"
8          level = 12
9      elif period_name == "weekend":
10         severity = "HIGH"
11         level = 12
12     else:
13         severity = "MEDIUM"
14         level = 9
15
16     alert = {
17         "timestamp": datetime.utcnow().isoformat(),
18         "rule": {
19             "id": "118725",
20             "level": level,
21             "description": f"{severity} - DATA EXFILTRATION [{
22                 period_name.upper()}]: "
23                 f"{data['total_octets']:,} octets ({total_mb
24                     :.1f}MB) "
25                 f"exfiltrated from {src_ip} to {len(data['
26                     destinations'])} destinations "
27                 f"in the last {TIME_WINDOW_MINUTES} minutes "
28                 f"across {data['connections']} connections "
29                 f"(threshold: {threshold_mb:.0f}MB) "
30         },
31         "agent": {"name": "sophos-integration"},
32         "data": {
33             "src_ip": src_ip,
34             "total_octets": data["total_octets"],
35             "total_mb": round(total_mb, 2),
36             "threshold_mb": round(threshold_mb, 2),
37             "period": period_name,
38             "connections": data["connections"],
39             "destinations": list(data["destinations"]),
40             "time_window_minutes": TIME_WINDOW_MINUTES
41         }
42     }
43
44     with open(ALERT_FILE, "a") as f:
45         f.write(json.dumps(alert) + "\n")

```

FIGURE 52 – Script sophos-exfiltration.py -3eme partie

```

1 def main():
2     try:
3         threshold_octets, period_name = get_current_threshold()
4
5         now = datetime.now()
6         print(f"[{now.strftime('%Y-%m-%d %H:%M:%S')}]")
7         print(f"   Period: {period_name}")
8         print(f"   Threshold: {threshold_octets:,} octets ({
          threshold_octets/1_000_000:.0f}MB)")
9         print(f"   Time window: {TIME_WINDOW_MINUTES} minutes")
10        print()
11
12        token = get_token()
13        events_data = get_events(token)
14        events = events_data.get("data", {}).get("hits", {}).get("hits
          ", [])
15
16        print(f"   Events found: {len(events)}")
17
18        if len(events) == 0:
19            print("   No events to analyze.")
20            return
21
22        ip_totals = analyze_exfiltration(events)
23
24        alerts_generated = 0
25        for src_ip, data in ip_totals.items():
26            total_mb = data['total_octets'] / 1_000_000
27
28            if data["total_octets"] >= threshold_octets:
29                generate_alert(src_ip, data, threshold_octets,
          period_name)
30                alerts_generated += 1
31                print(f"ALERT: {src_ip} - {data['total_octets']:,}
          octets ({total_mb:.1f}MB) - EXCEEDS THRESHOLD")
32            else:
33                print(f"OK: {src_ip} - {data['total_octets']:,} octets
          ({total_mb:.1f}MB) - below threshold")
34
35            print(f"\n   Total alerts generated: {alerts_generated}")
36
37        except Exception as e:
38            print(f"ERROR: {e}", file=sys.stderr)
39            sys.exit(1)
40
41 if __name__ == "__main__":
42     main()

```

FIGURE 53 – Script sophos-exfiltration.py -4eme partie

Graphiques d'analyse de production

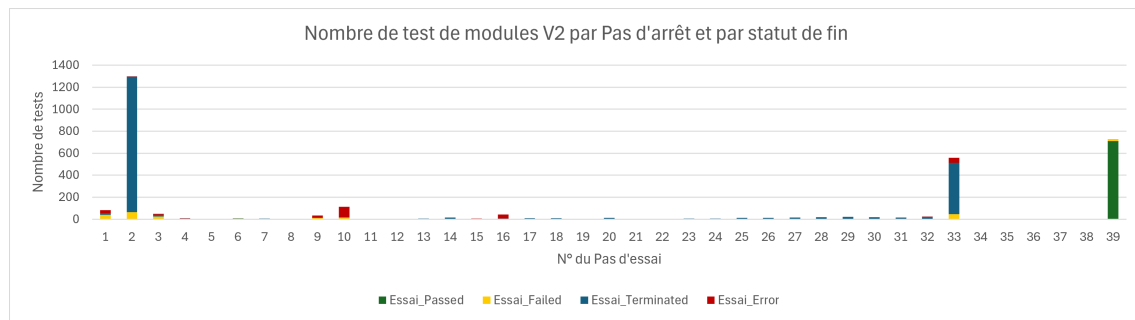
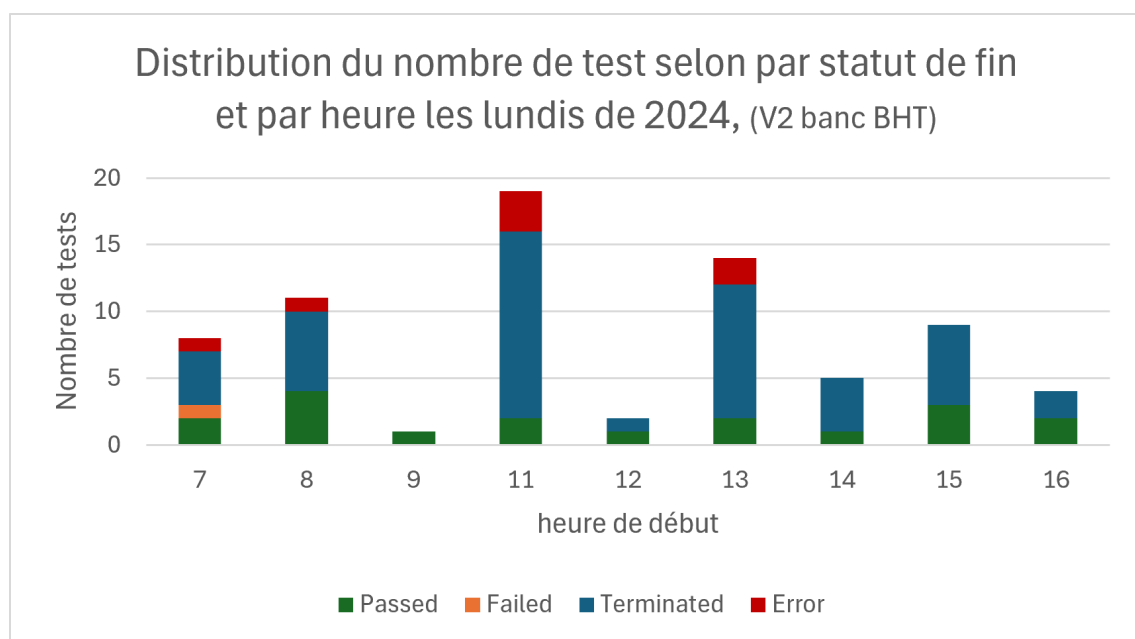
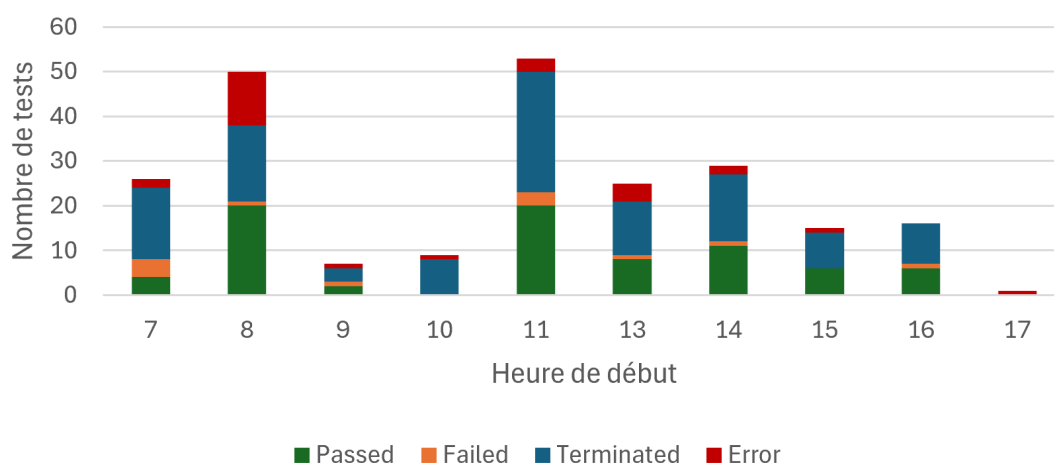


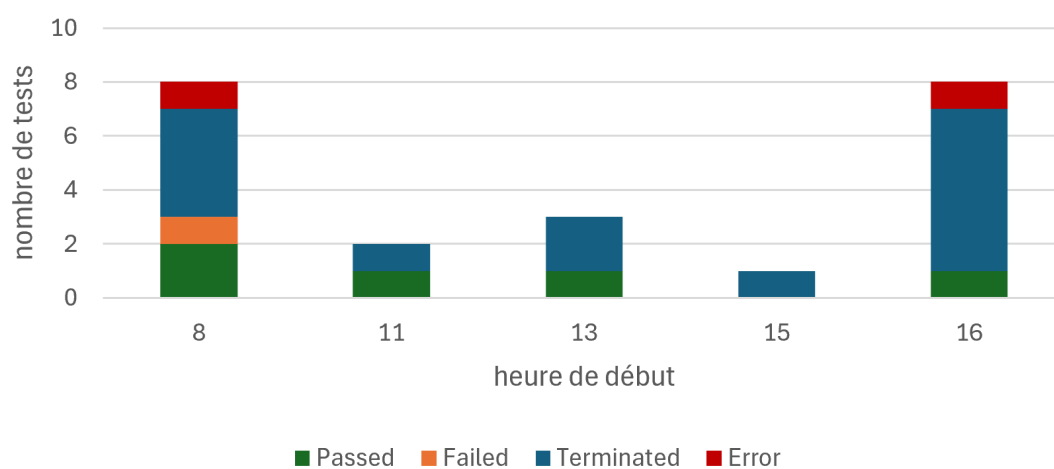
FIGURE 54 – Distribution des statuts de fin des essais par étape de l'essai, modules V2



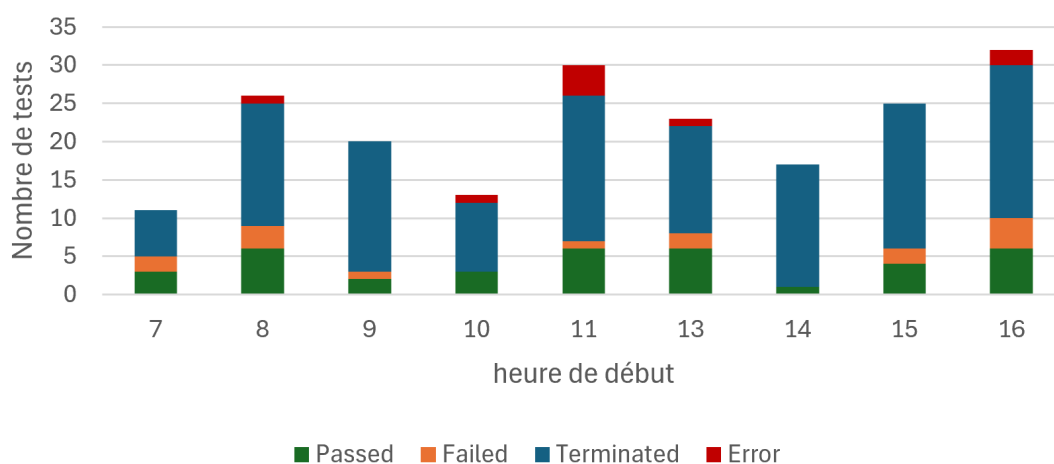
Distribution du nombre de test selon par statut de fin
et par heure les lundis de 2025, (V2 banc BHT)



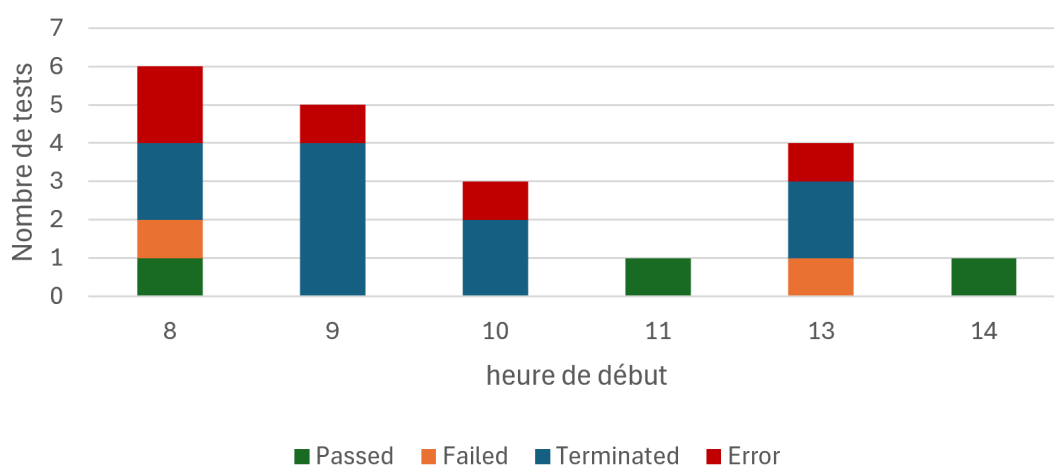
Distribution du nombre de test selon par statut de fin
et par heure les lundis de 2026, (V2 banc BHT)



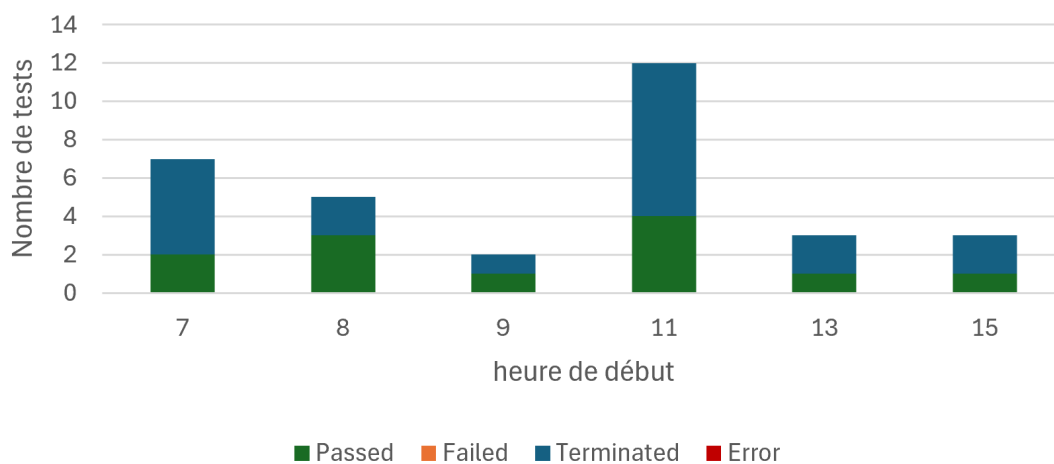
Distribution du nombre de test selon par statut de fin
et par heure les lundis de 2024, (Modules V2 banc B1)



Distribution du nombre de test selon par statut de fin
et par heure les lundis de 2025, (Modules V2 banc B1)



Distribution du nombre de test selon par statut de fin
et par heure les lundis de 2026, (Modules V2 banc B1)



Références bibliographiques

Références

- [1] Center for Internet Security (CIS), [CIS Benchmarks](#), consulté en 2026.
- [2] Wazuh, [Wazuh Documentation](#), consulté en 2026.
- [3] Microsoft. (2024). [Security baselines for Windows](#), consulté en 2026.
- [4] Microsoft. (2024). [Windows Server security documentation](#), consulté en 2026.
- [5] Microsoft. (2024). [Active Directory Domain Services overview](#), consulté en 2026.
- [6] Microsoft. (2024). [Best practices for securing Active Directory](#), consulté en 2026.
- [7] Microsoft. (2024). [Group Policy documentation](#), consulté en 2026.
- [8] Wazuh. (2024). [Ruleset documentation](#), consulté en 2026.
- [9] Wazuh. (2024). [Active response documentation](#), consulté en 2026.
- [10] Wazuh. (2024). [Wazuh agent documentation](#), consulté en 2025.
- [11] ANSSI. (2024). *Guide d'hygiène informatique*, consulté en 2026.
- [12] ANSSI. (2024). *Administration sécurisée des systèmes d'exploitation*, consulté en 2026.
- [13] ISO/IEC. (2022). *ISO/IEC 27002 :2022 — Information security, cybersecurity and privacy protection — Information security controls*, consulté en 2026.
- [14] MITRE. (2024). [ATT&CK for enterprise](#), consulté en 2026.
- [15] Cybersecurity and Infrastructure Security Agency. (2024). [Known exploited vulnerabilities catalog](#), consulté en 2026.
- [16] IT-Connect. (2026). [Divers tutoriels de Florian BURNEL](#), consulté en 2026.
- [17] Ressources internes à Mesulog (2026). Schémas de présentation, consulté en 2026.
- [18] **Utilisation de l'IA : l'IA n'a pas été utilisée pour produire du texte, mais a été utilisée pour corriger les fautes d'orthographe et assister à la création de certains schémas.**